

第7章 查找

教 学 内 容

- 7.1 查找的基本概念
- 7.2 线性表的查找(静态查找表)
 - 7.2.1 顺序查找(顺序表的查找)
 - 7.2.2 折半查找(有序表的查找)
 - 7.2.3 分块查找
- 7.3 树表的查找(动态查找表)
 - 7.3.1 二叉排序树
 - 7.3.2 平衡二叉树
 - 7.3.3&7.3.4 B-树 B+树
- 7.4 散列表的查找

查找的基本概念(P230)

查找表：是由同一类型的数据元素（或记录）构成的集合，而每个记录由若干个数据项组成。

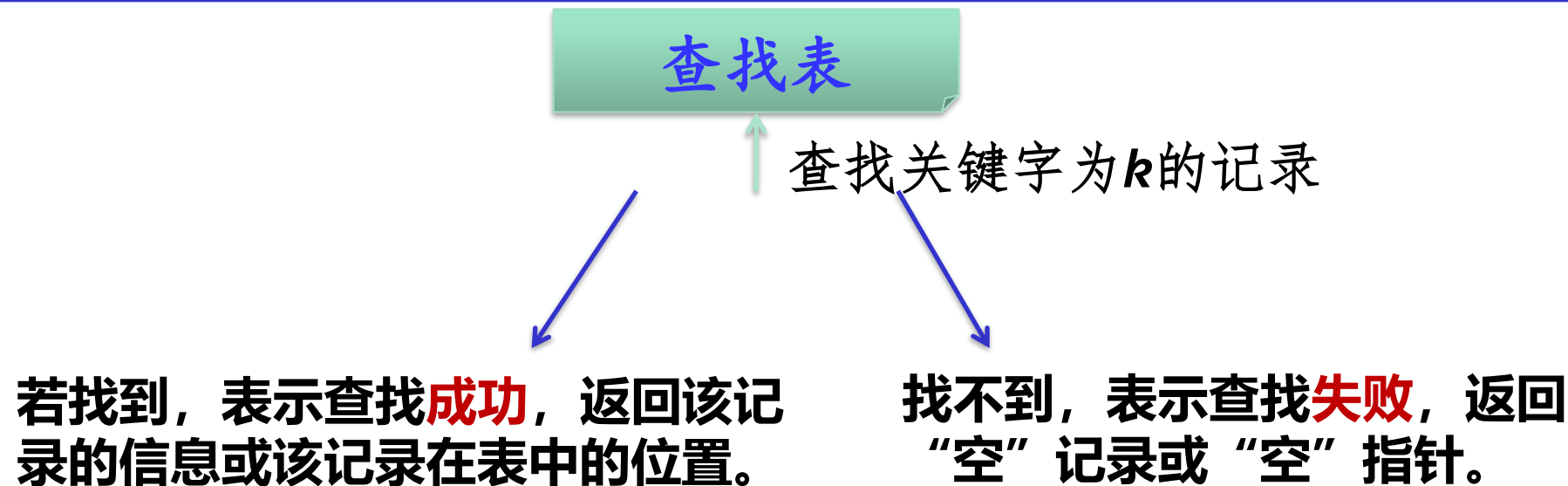
查找：在查找表中找出某个“特定的”数据元素（或记录）。

关键字：数据元素（或记录）中某个数据项的值，用它标识该数据元素（或记录）。

若此关键字可以唯一标识一个记录，那就称为**主关键字**，而用以识别若干记录的关键字为**次关键字**。

[illegible]

查找的基本概念(P230)



对查找表经常进行的操作：

- 1、**查询**某个“**特定的**”数据元素是否在查找表中；
- 2、**检索**某个“**特定的**”数据元素的各种属性；
- 3、在查找表中**插入**一个数据元素；
- 4、从查找表中**删去**某个数据元素。

静态查找表

动态查找表

查找算法的性能分析(P231)

查找运算时间主要花费在关键字比较上，通常把查找过程中需和给定值进行比较的关键字个数的期望值称为查找算法在查找成功时的**平均查找长度**，衡量一个查找算法效率优劣的标准。

平均查找长度ASL (Average Search Length) 定义为：

关键字的平均比较次数 $ASL = \sum_{i=1}^n p_i c_i$

- n 是查找表中数据元素（记录）的个数。
- p_i 是查找第*i*个记录的**概率**，一般地，认为每个记录的查找概率相等，即等概率， $p_i=1/n$ ($1 \leq i \leq n$)。
- c_i 是找到第*i*个记录所需的**比较次数**（或者是已比较过的关键字个数）。

7.2 顺序表的查找(P231)

提纲

CONTENTS

7.2.1 顺序查找

7.2.2 折半查找

7.2.3 分块查找

【静态查找表的元素类型定义】(P231)

```
typedef float KeyType;           //关键字(成绩)的数据类型为实型
typedef int KeyType;             //关键字(学号)的数据类型为整型
typedef char *KeyType;           //关键字(姓名)的数据类型为字符串型

typedef struct {                 //对数据元素进行类型定义
    KeyType key;                 //关键字域，关键字的数据类型
    InfoType otherinfo;         //其他信息域
} SElemType;                    //查找Search顺序表元素类型

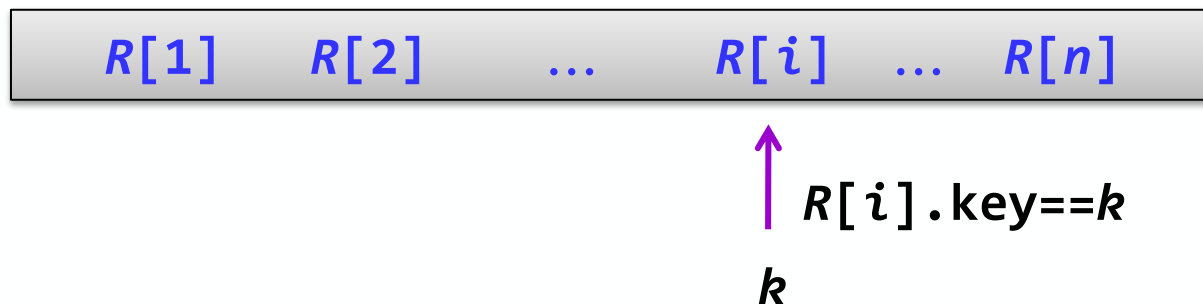
typedef struct {                //对顺序表的结构类型定义
    SElemType *R;               //存储数组空间基址 R[0]的地址
    int length;                 //当前表长
} SSTable;                     //Sequential Search Table
```

```
SSTable ST; //定义顺序表ST
```

7.2.1 顺序表的查找(P231)

顺序查找

思路：从表的一端开始，顺序扫描线性表，挨个将扫描到的关键字和给定值 k 比较：



- 若当前扫描到的关键字与 k 相等，则查找成功，返回位置 i
- 若扫描结束，仍未找到关键字等于 k 的记录，则查找失败，返回 0

【顺序查找算法描述】 (P232)

```
int Search_Seq(SSTable ST, KeyType k) {
```

//在顺序表ST中的最右端开始，顺序查找其关键字为k的数据元素。若查找成功返回其在表中的位置，否则返回0

```
    for(int i=ST.length; i>=1; --i) {
```

```
        if(ST.R[i].key==k)
```

```
            return i;
```

```
    }
```

```
    return 0; //查找失败返回0
```

```
}
```


【顺序查找算法描述】 (P232)

法2：设置哨兵，将待查找关键字k存储在顺序表的0号位置，从后往前查找，避免每次都去判断是否查找结束，提高查找速度

```
int Search_Seq(SSTable ST, KeyType k) {
```

//在顺序表ST中顺序查找其关键字为k的数据元素。若查找成功返回其在表中的位置，否则返回0

```
    ST.R[0].key=k;    //监视哨
```

```
    for(int i=ST.length; ST.R[i].key!=k; --i) ; //从后往前判断是否等于给定值k，满足等于跳出for循环
```

k=20

```
    return i;
```

```
}
```

0	1	2	3	4	5	6	7
20	5	6	7	8	9	10	11

7.2.1 顺序查找(P232)

$$ASL = \sum_{i=1}^n p_i c_i$$

① 成功情况下的平均查找长度— $ASL_{\text{成功}}$

■ 有 n 个元素，等概率时 $p_i=1/n$ 。

■ 因此成功时的顺序查找的平均查找长度为：

$$ASL_{\text{成功}} = 1p_n + 2p_{n-1} + 3p_{n-2} + \dots + (n-1)p_2 + np_1$$

$$= \frac{1}{n}(1+2+\dots+n)$$

$$= \frac{1}{n} \times \frac{(1+n)n}{2} = \frac{n+1}{2}$$

时间复杂度： $O(n)$



查找成功时的平均比较次数约为表长的一半。

7.2.1 顺序表的查找

② 不成功情况下的平均查找长度— $ASL_{\text{不成功}}$

- 查找不成功时需要扫描完整张表，与表中所有元素都比较一次。
- $ASL_{\text{不成功}} = n$ 。



顺序查找的时间复杂度为 $O(n)$ 。

顺序查找分析(P232)

优点:

算法简单，无需排序，采用顺序存储和链式存储都可以，对数据元素关键字内容的有序性没有要求。

缺点:

- (1) 顺序查找的时间复杂度为 $O(n)$
- (2) 当 n 很大时，平均查找长度ASL较大，不宜采用顺序查找

7.2.2 有序表的查找(P232)

折半查找

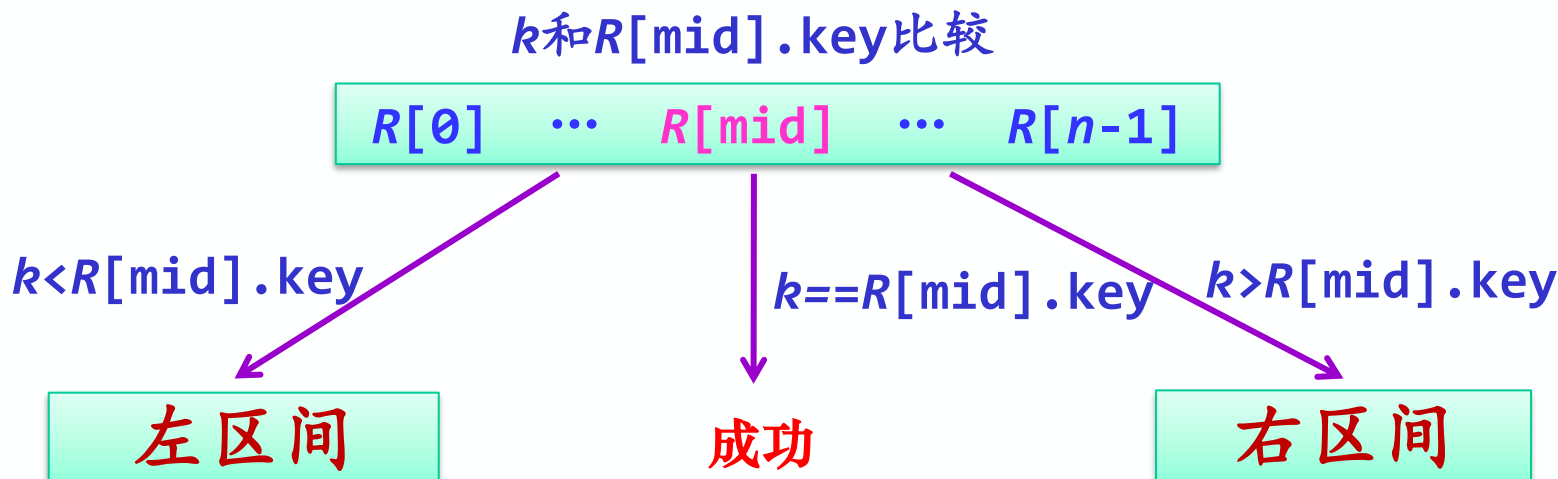
递归

折半查找又称为二分查找，是一种效率较高的查找方法。

前提条件：要求静态查找表必须是有序表，即表中记录已按关键字排序。

查找思想(假设有序表关键字**升序**排列)：

- (1) 从表的**中间记录**开始，若**给定值** k == **中间值关键字**，则查找成功；
- (2) 若给定值 k > 中间值，则在表中大于中间值的那一半中查找
- (3) 若给定值 k < 中间值，则在表中小于中间记录的那一半中查找



指针low: 待查区间的下界	若 $k == R[mid].key$, 查找成功
指针high: 待查区间的上界	若 $k < R[mid].key$, 则 $high = mid - 1$
指针mid: 指向区间的中间位置	若 $k > R[mid].key$, 则 $low = mid + 1$
初值为 $low = 0, high = n - 1, mid = \lfloor (low + high) / 2 \rfloor$	

若 $k == R[mid].key$, 查找成功

若 $k < R[mid].key$, 则 $high = mid - 1$

若 $k > R[mid].key$, 则 $low = mid + 1$

1	2	3	4	5	6	7	8	9	10	11
05	13	19	21	37	56	64	75	80	88	92

low
mid
high

```
low=1      high=11
mid=⌊(low + high)/2⌋=6
```

21 < R[mid].key (21 < 56)
low=1 high=mid-1=5
mid = ⌊(low + high)/2⌋ = 3

```
21>R[mid].key (21>19)
low=mid+1=4      high=5
mid=|(low + high)/2|=4
```

三次比较，查找21成功

查找70:

若 $k == R[mid].key$, 成功

若 $k < R[mid].key$, 则 $high = mid - 1$

若 $k > R[mid].key$, 则 $low = mid + 1$

1	2	3	4	5	6	7	8	9	10	11
5	13	19	21	37	56	64	75	80	88	92
↑ low		↑ mid				↑ high				

初始状态:

low=1 high=11

mid=**⌊(low + high)/2⌋**=6

1	2	3	4	5	6	7	8	9	10	11
5	13	19	21	37	56	64	75	80	88	92

low **mid** **high**

70>R[mid].key (70>56)

low=mid+1=7 high=11

mid=**⌊(low + high)/2⌋**=9

1	2	3	4	5	6	7	8	9	10	11
5	13	19	21	37	56	64	75	80	88	92

↑
↑
↑
low mid high

70 < R[mid].key (70 < 80)

low=7 high=mid-1=8

mid=**⌊(low + high)/2⌋**=7

查找70:

若 $k == R[mid].key$, 成功

若 $k < R[mid].key$, 则 $high = mid - 1$

若 $k > R[mid].key$, 则 $low = mid + 1$

1	2	3	4	5	6	7	8	9	10	11
5	13	19	21	37	56	64	75	80	88	92

↑ ↑ ↑
low | high
mid

$70 > R[mid].key$ ($70 > 64$)

$low = mid + 1 = 8$ $high = 8$

$mid = \lfloor (low + high) / 2 \rfloor = 8$

1	2	3	4	5	6	7	8	9	10	11
5	13	19	21	37	56	64	75	80	88	92

↑ ↑ ↑
high | low
mid

$70 < R[mid].key$ ($70 < 75$)

$low = 8$ $high = mid - 1 = 7$

$mid = \lfloor (low + high) / 2 \rfloor = 7$

直至 $low > high$ 时, 查找失败

折半查找示例2

在11位关键字有序序列 (2, 3, 10, 15, 20, 25, 28, 29, 30, 35, 40) 中采用折半查找法查找关键字为15的元素。

初始状态: low=1 high=11 mid= $\lfloor (\text{low} + \text{high})/2 \rfloor = 6$

关键字序列: 2 3 10 15 20 25 28 29 30 35 40

low mid 关键字比较1次 high

关键字序列: 2 3 10 15 20 25 28 29 30 35 40

low mid high

关键字比较2次

关键字序列: 2 3 10 15 20 25 28 29 30 35 40

low mid high

关键字比较3次查找成功, 记录的下标是3, 逻辑序

关键字比较3次查找成功，关键字为15的记录的下标是3，逻辑序号为4。

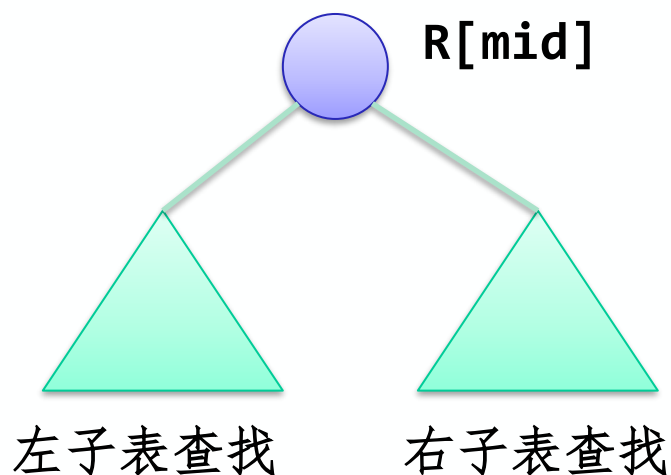
【折半查找算法描述】 (P233)

```
int Search_Bin(SSTable ST, KeyType key){  
    /*在有序表ST中查找等于key的数据元素，若找到，则函数值为该元素  
    在表中的位置，否则查找失败，返回0*/  
    low=1; high=ST.length;    //置查找区间初值  
    while(low<=high) //查找区间大小大于0，可以不断查找  
    {  
        mid=(low+high)/2; //mid指向查找区间的中间位置  
        if(key==ST.R[mid].key)    return mid; //找到待查元素  
        else if(key<ST.R[mid].key)    high=mid-1; //左半区间继续查找  
        else    low=mid+1;    //右半区间继续查找  
    }  
    return 0;    //表中不存在待查元素，查找失败  
}
```

折半查找判定树(详见教材P234)

折半查找过程可用二叉树来描述：

- 把当前查找区间的中间位置上的记录作为**根**；
- 左子表和右子表中的记录分别作为根的**左子树**和**右子树**。

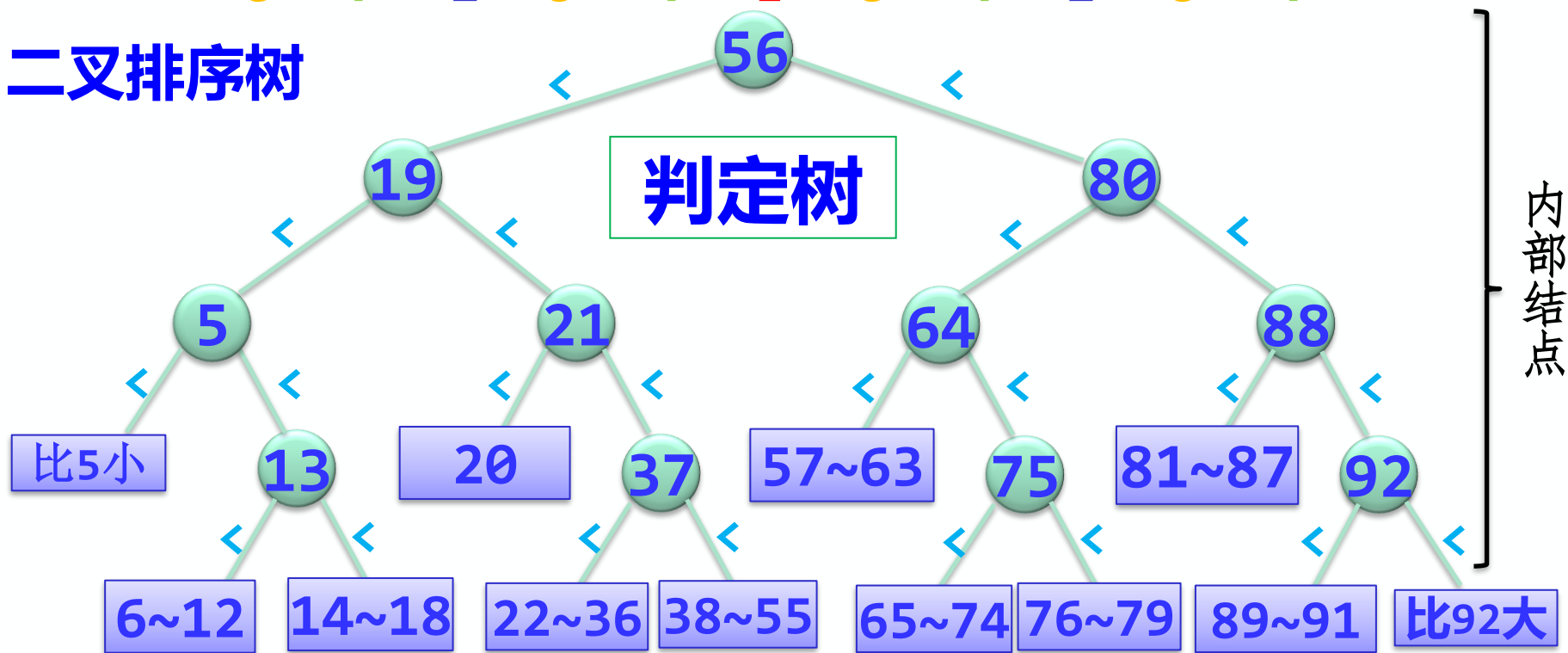


这样的二叉树称为**判定树**或**比较树**。

折半查找判定树(详见教材P234)

1	2	3	4	5	6	7	8	9	10	11
5	13	19	21	37	56	64	75	80	88	92
3	4	2	3	4	1	3	4	2	3	4

二叉排序树



内部结点即查找成功对应的结点，比较次数=路径上的结点数=该结点所在层次
比较次数 $\leq \lfloor \log_2 n \rfloor + 1$ (树的深度)

外部结点即查找失败对应的结点，比较的次数=路径上的内部结点数=该结点所在层次-1

n 个关键字：内部结点为 n 个，外部结点为 $n+1$ 个

折半查找

【例】 对于给定11个数据元素的有序表：

(2, 3, 10, 15, 20, 25, 28, 29, 30, 35, 40)

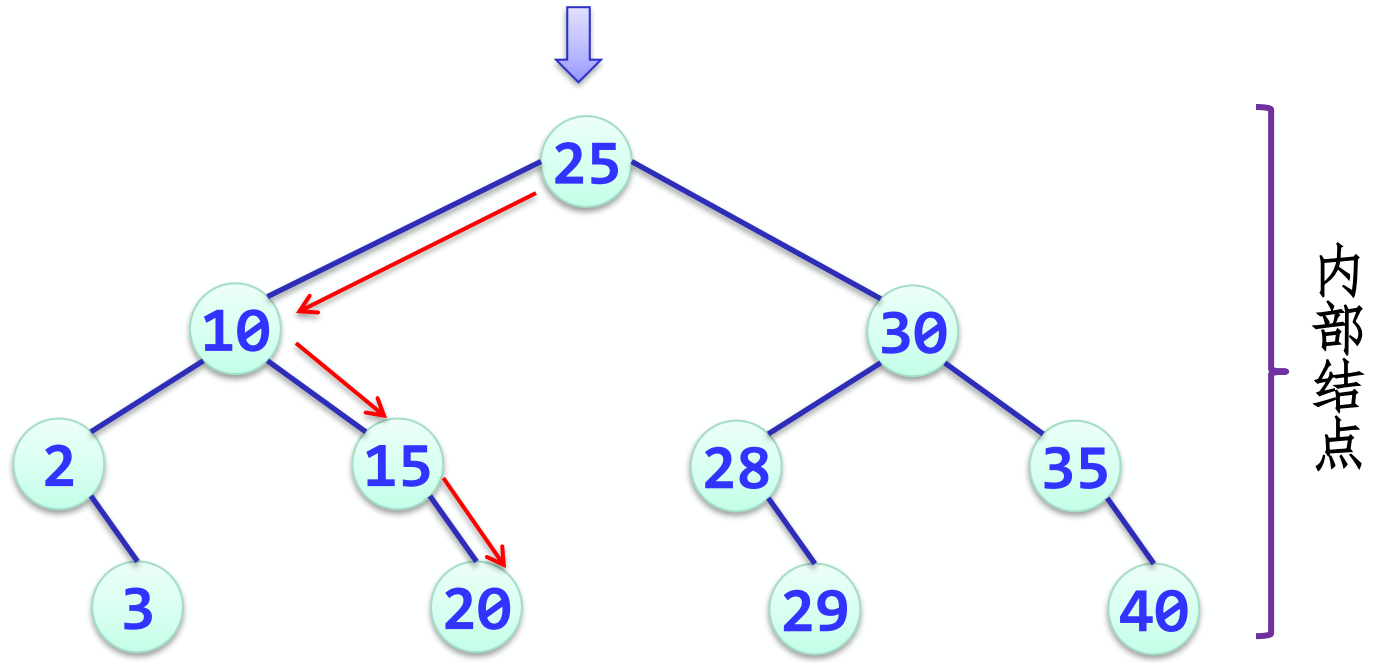
采用折半查找，请问：

- (1) 若查找给定值为20的元素，将依次与表中哪些元素比较？
- (2) 若查找给定值为26的元素，将依次与表中哪些元素比较？
- (3) 假设查找表中每个元素的概率相同，求查找成功时的平均查找长度和查找不成功时的平均查找长度。

折半查找

1 2 3 4 5 6 7 8 9 10 11
(2, 3, 10, 15, 20, 25, 28, 29, 30, 35, 40)

判定树:

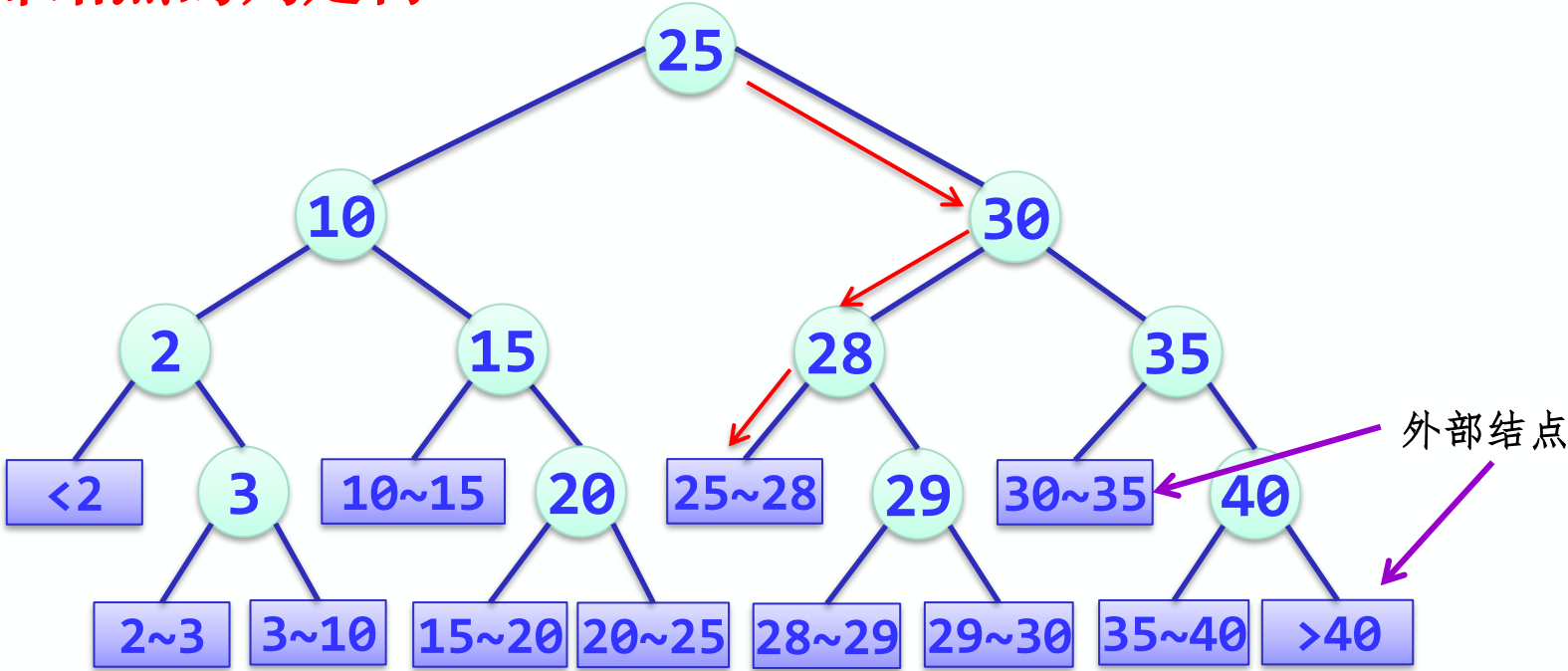


解：若查找给定值为20的元素，依次与表中25、10、15、20元素比较，共比较4次。

折半查找成功 $ASL_{成功} = \sum_{i=1}^n p_i c_i = \frac{1 \times 1 + 2 \times 2 + 4 \times 3 + 4 \times 4}{11} = 3$

折半查找

带外部结点的判定树：

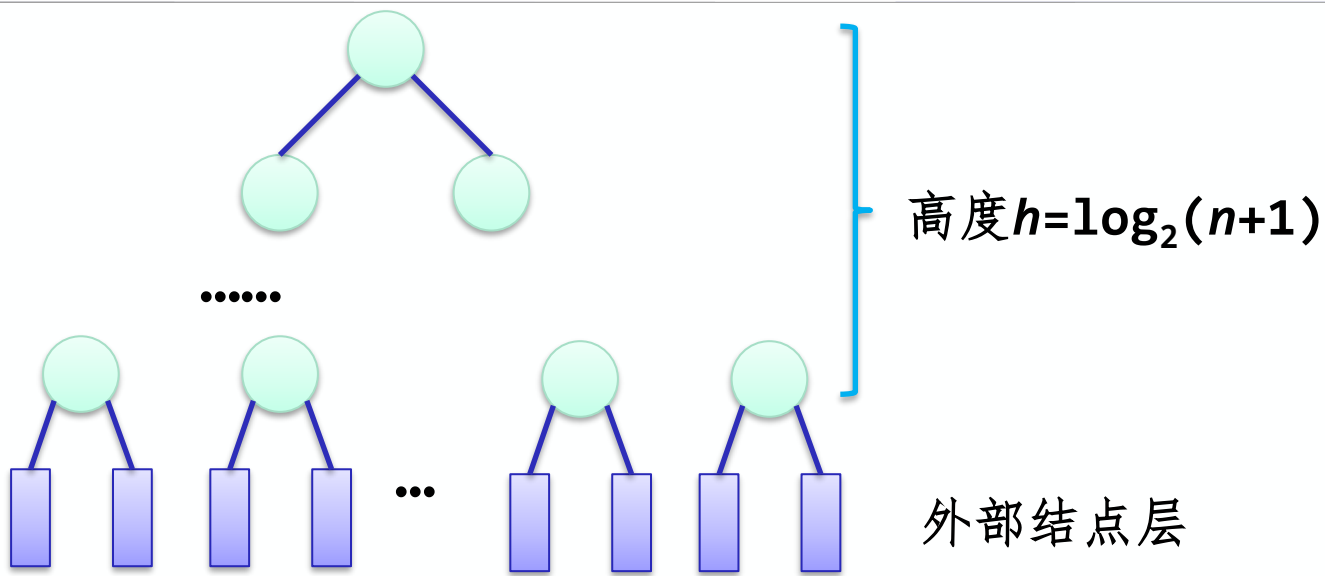


解：若查找给定值为**26**的元素，依次与25、30、28元素比较，共比较3次。

折半查找失败 $ASL_{\text{不成功}} = \frac{4 \times 3 + 8 \times 4}{12} = 3.67$

折半查找

- 内部结点总数 $n=2^h - 1$ ，则 $h=\log_2(n + 1)$ （此时判定树为满二叉树）
- 树中第 j 层上结点数为 2^{j-1} ，查找该层上的每个记录需要进行 j 次比较。
- 有 n 个元素，等概率时 $p_i=1/n$ 。



折半成功时的平均查找长度为：

$$ASL_{bn} = \sum_{i=1}^n p_i c_i = \frac{1}{n} \sum_{j=1}^h 2^{j-1} \times j = \frac{n+1}{n} \log_2(n+1) - 1 \approx \log_2(n+1) - 1$$

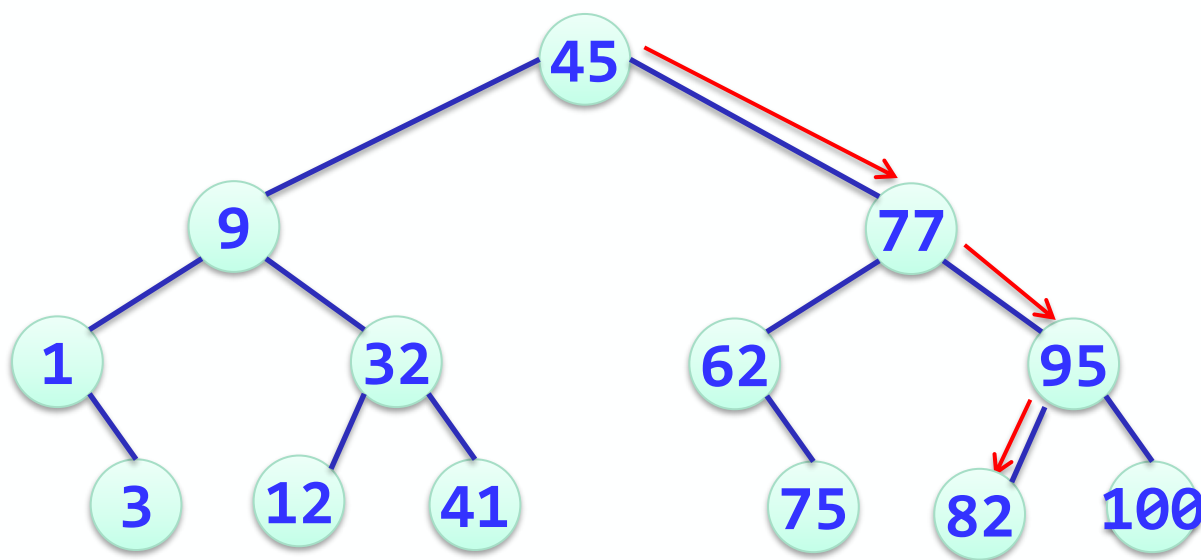
【练习1】 有一个有序表为{1,3,9,12,32,41,45,62,75,77,82,95,100}, 当折半查找值82的元素时, () 次比较后查找成功。

A. 1

B. 2

C. 3

D. 4



答案: D

【练习2】下列选项中，不能构成折半查找过程中关键字比较序列的是（ ）。

A. 500, 450, 200, 180

B. 180, 500, 200, 450

C. 180, 200, 500, 450

D. 500, 200, 450, 180

A.

180 200 450 500

B.

180 200 450 500

C.

180 200 450 500

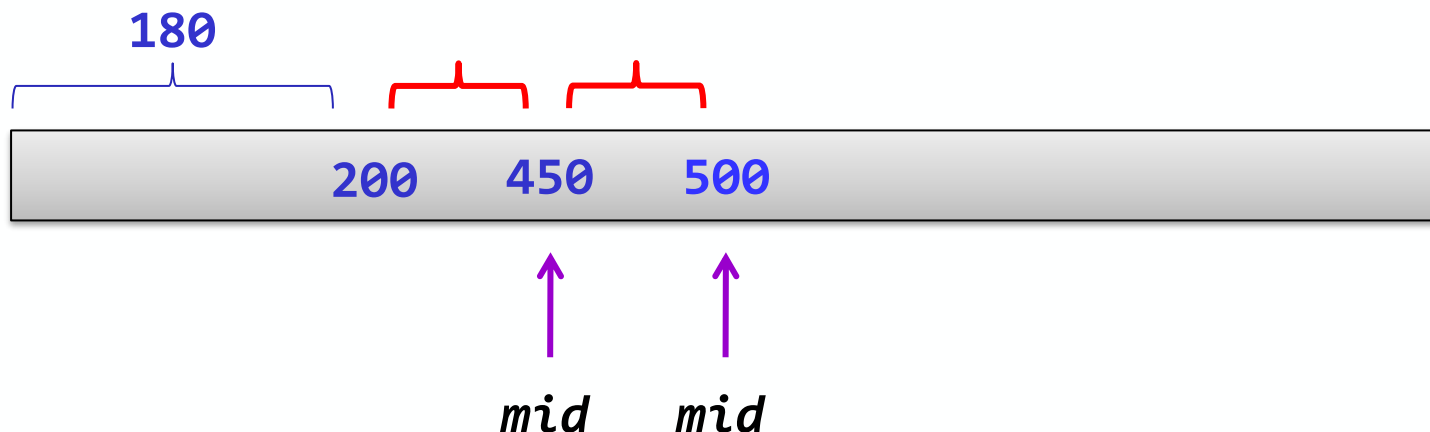
【练习2】下列选项中，不能构成折半查找过程中关键字比较序列的是（ ）。

A. 500, 450, 200, 180

B. 180, 500, 200, 450

C. 180, 200, 500, 450

D. 500, 200, 450, 180



答案：D

折半查找分析

优点：线性表作为查找表的组织形式，折半查找的效率比顺序查找高。

特点：每次将待查记录所在区间缩短一半

局限性：

- (1) 要求线性表中记录按关键字有序排序
- (2) 且不能用链表做存储结构（对线性链表无效）
- (3) 适用于查找频率高，元素改动少的有序表中

总结：线性表的查找更适用于静态查找表，若要对动态查找更高效率的查找，可以用特殊的二叉树作为查找表的组织形式。

7.2.3 分块查找——块间有序，块内无序(P235)

分块有序，即分成若干子表（块），要求每个块中的数值都比后一个块中数值小（但块内未必有序）。

对每个块建立一个**索引表**，其中要包含关键字项（每个块中值最大的关键字）和指针项（该块的起始地址）。

查找：38

$22 < 38 < 48$

return 10

索引表

22	48	86
1	7	13

块内最大关键字

第一个记录在整个表中的起始地址

22	12	13	8	9	20	33	42	44	38	24	48	60	58	74	49	86	53
----	----	----	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----

第1块

第2块

第3块

- ① 对索引表使用折半查找法（因为索引表是有序表）；
- ② 确定了待查关键字所在块后，在块内采用顺序查找法（因为各块内部是无序的）；

7.2.3 分块查找——块间有序，块内无序(P236)

查找效率： $ASL=L_b+L_w$

对索引表折半查找的ASL

对块内顺序查找的ASL

优点：插入和删除比较容易，无需进行大量移动。

缺点：要增加一个索引表的存储空间并对初始索引表进行排序运算。

适用情况：如果线性表既要快速查找，又经常动态变化，则可采用分块查找。

7.3 动态查找表(P237)

动态查找表的特点是：

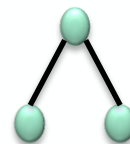
表结构本身是在查找过程中动态生成的，即对于给定值key，若表中存在其关键字等于key的记录，则查找成功返回，否则插入关键字等于key的记录。

动态查找表不仅适合于查找，也适合于**插入和删除**操作。

常见的树表：

- 二叉排序树
- 平衡二叉树

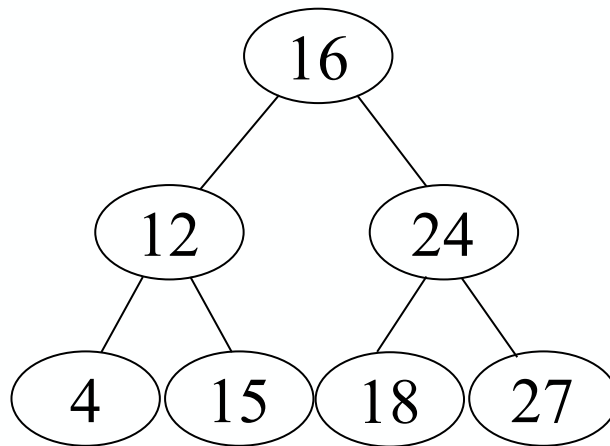
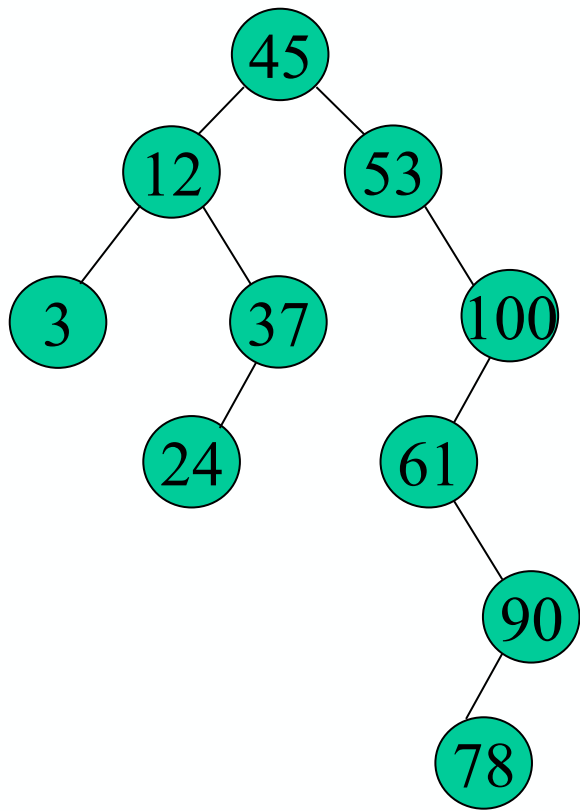
} 二叉树



7.3.1 二叉排序树 (P237)

二叉排序树 (BST) 或是空树，或是满足如下性质的二叉树：

- (1) 若其左子树非空，则**左子树**上所有结点的值**均小于根结点的值**；
- (2) 若其右子树非空，则**右子树**上所有结点的值**均大于根结点的值**；
- (3) 其**左右子树本身又分别是二叉排序树** **递归**



二叉排序树

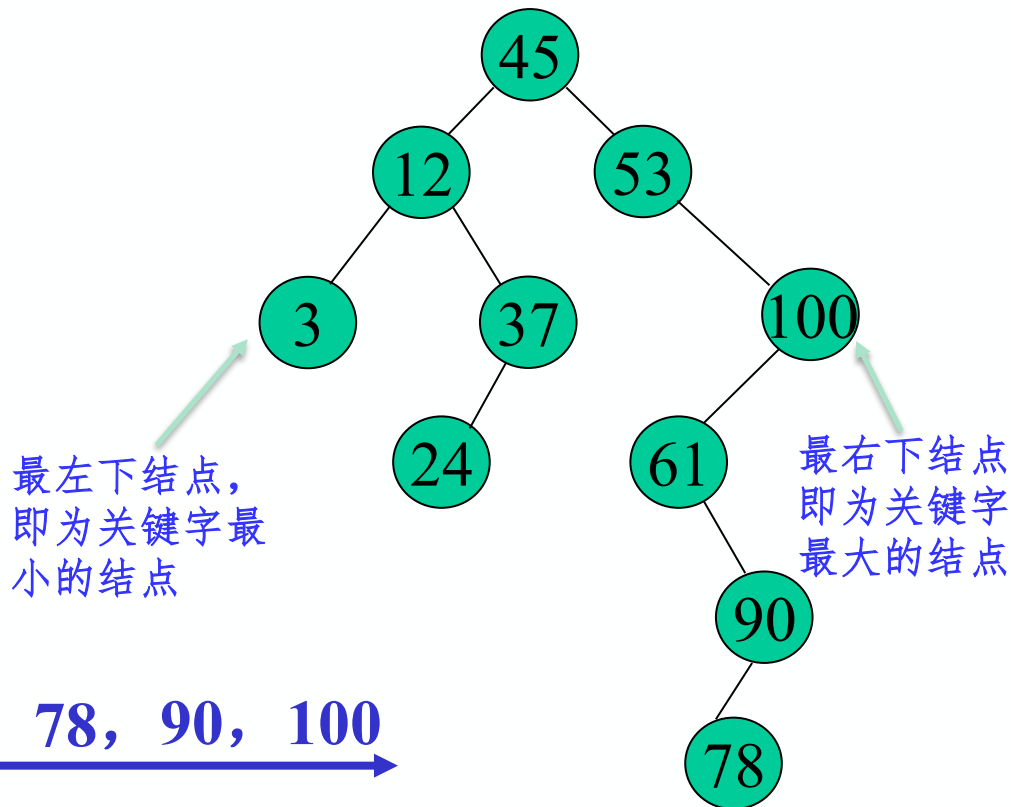
中序遍历二叉排序树(P237)

中序遍历二叉排序树后的结果有什么规律?

左根右

3, 12, 24, 37, 45, 53, 61, 78, 90, 100

递增



特点:

- 根结点的最左下结点是关键字最小的结点
- 根结点的最右下结点是关键字最大的结点

(1) 二叉排序树的操作 - 查找

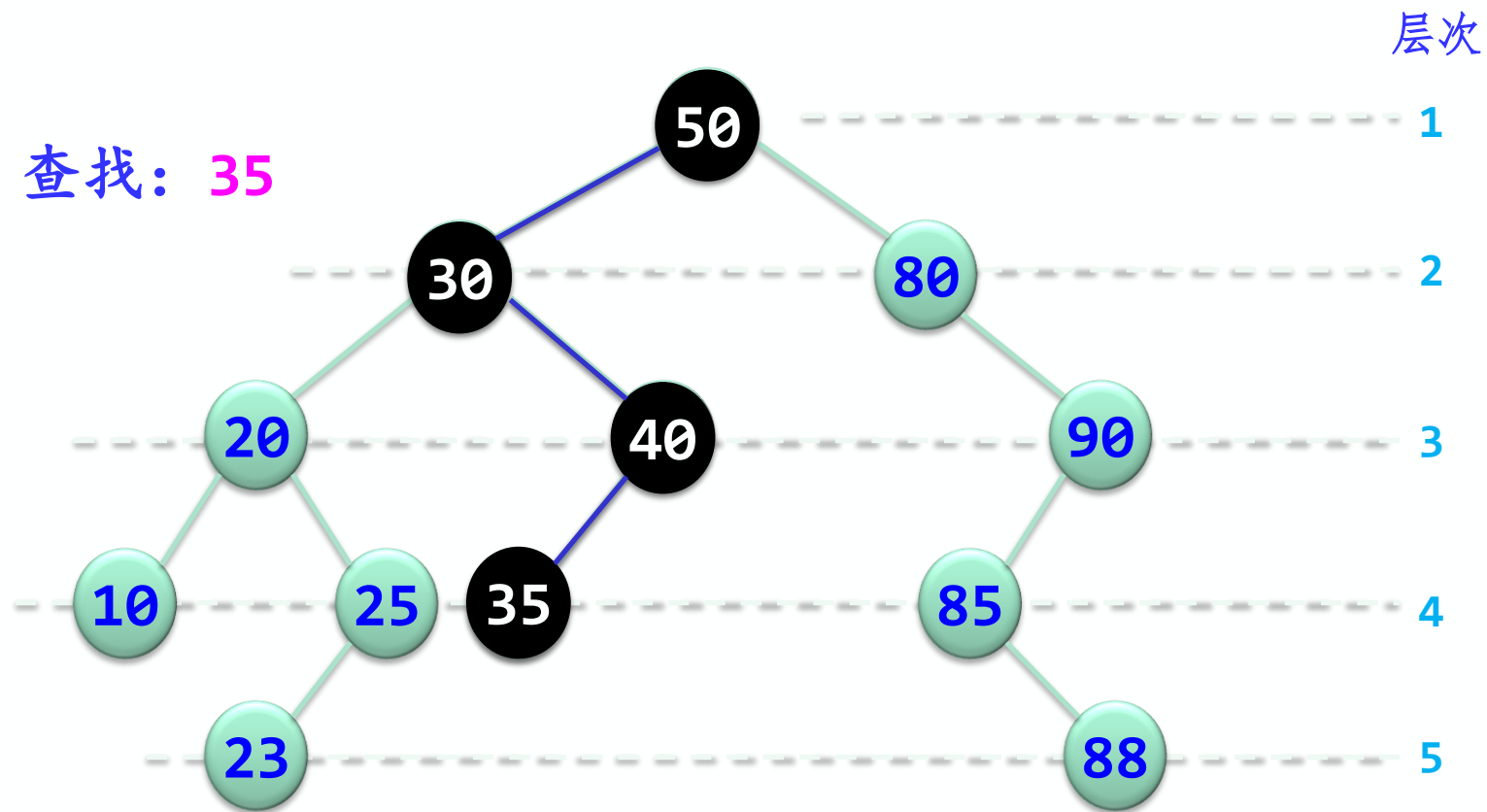
- 1、若二叉排序树为空，则查找失败。
- 2、若二叉排序树非空，从根结点开始查找。

将待查找的关键字与根结点进行比较：

- ①等于根结点，成功
- ②若小于根结点，查其左子树
- ③若大于根结点，查其右子树

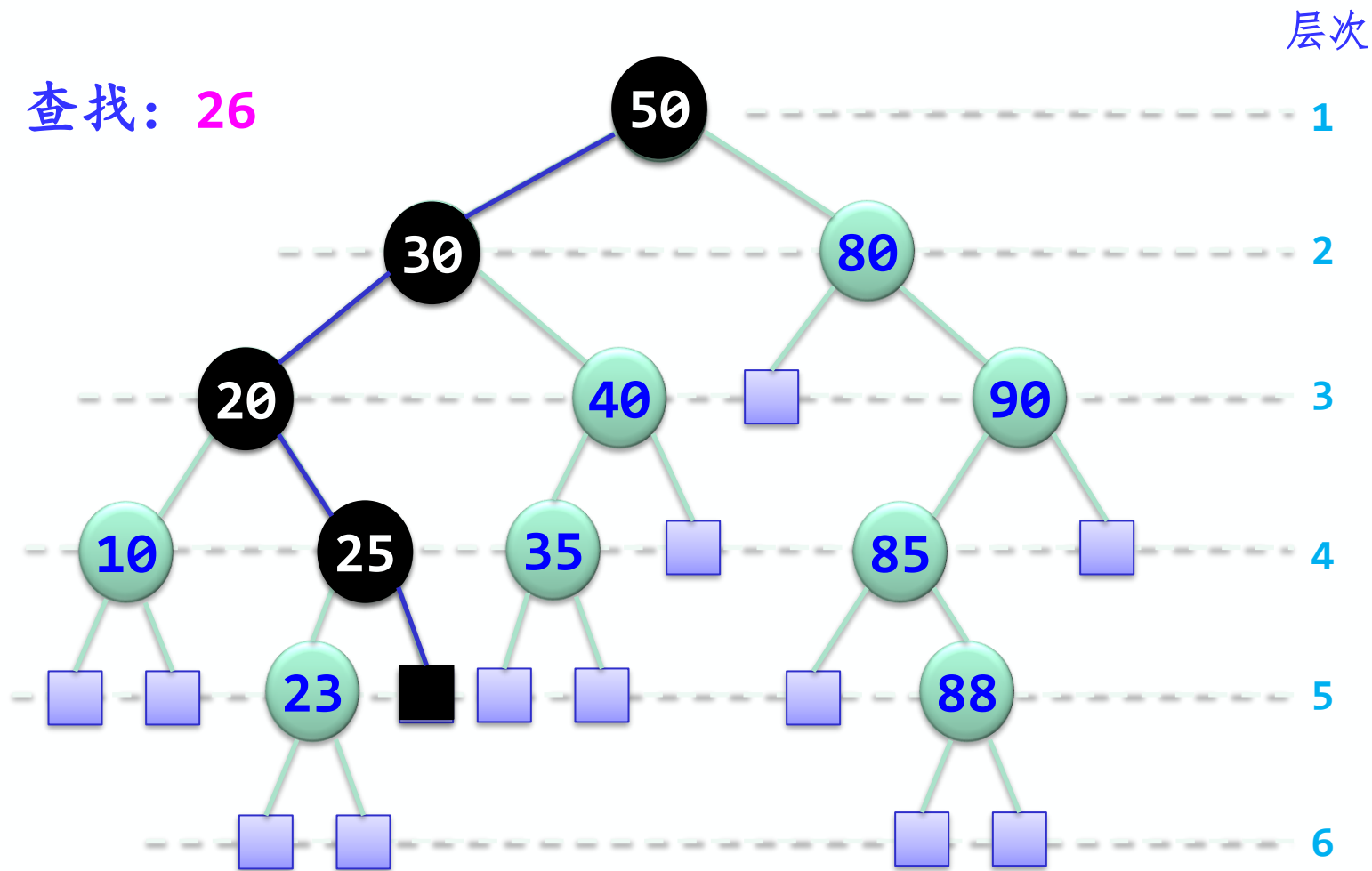
在左右子树上的操作类似

查找成功的时候



- 从根结点开始，查找最后落在某个内部结点，表明查找成功
- 比较的关键字次数=该内部结点所在层次=从根结点到该内部结点的路径上所有内部结点个数
- 查找路径是二叉排序树的一部分，本身也是一棵二叉排序树且一定是个连续的路径

查找失败的时候



- 从根结点开始，查找最后落在某个外部结点中
- 比较的关键字次数=该外部结点的层次-1=路径上内部结点的个数

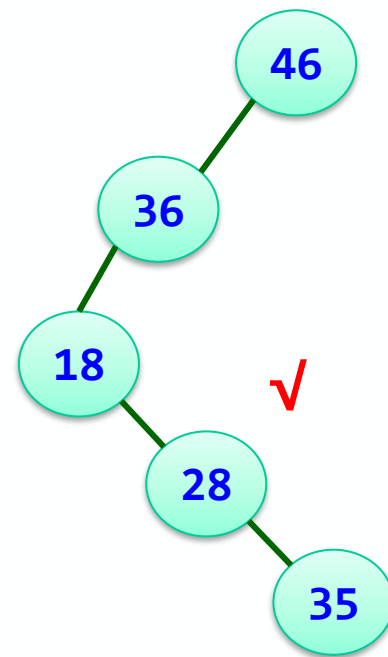
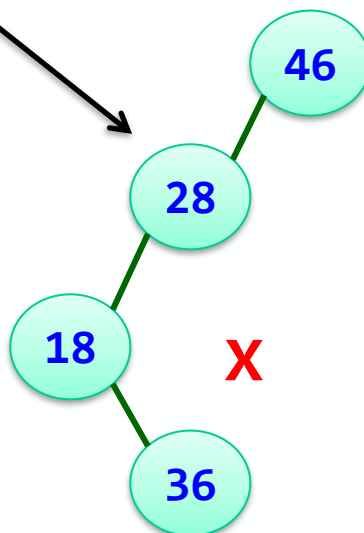
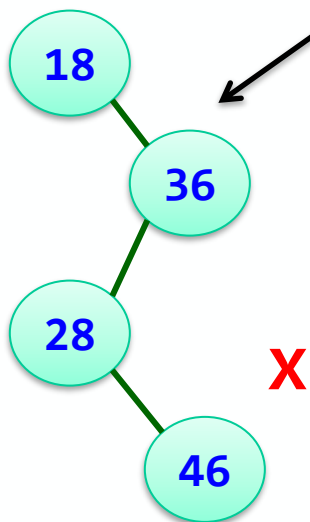
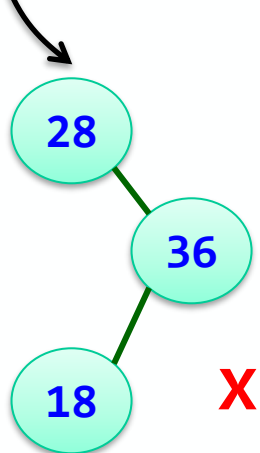
【练习3】在含有27个结点的二叉排序树上，查找关键字为35的结点，以下（ ）是可能的关键字比较序列？

A. 28, 36, 18, 46, 35

B. 18, 36, 28, 46, 35

C. 46, 28, 18, 36, 35

D. 46, 36, 18, 28, 35



答案为D

【二叉排序树的存储结构】

```
typedef float KeyType;           // 指定关键字的数据类型为实型
typedef int KeyType;             // 指定关键字的数据类型为整型
typedef char *KeyType;           // 指定关键字的数据类型为字符串型
```

```
typedef struct {                 // 数据元素的类型定义
    KeyType key;                 // 关键字域，关键字的数据类型
    InfoType otherinfo;         // 其他域
} ElemType;                     // 数据元素类型
```

```
typedef struct BSTNode{          // 二叉链表的存储形式
    ElemType data;               // 数据域
    struct BSTNode *lchild, *rchild; // 递归定义左右孩子指针
} BSTNode, *BSTree;             // 结点类型、指针类型
```

```
BSTree T; // 指针类型定义二叉排序树T，T指向根结点
```

【二叉排序树查找算法描述】 (P238)

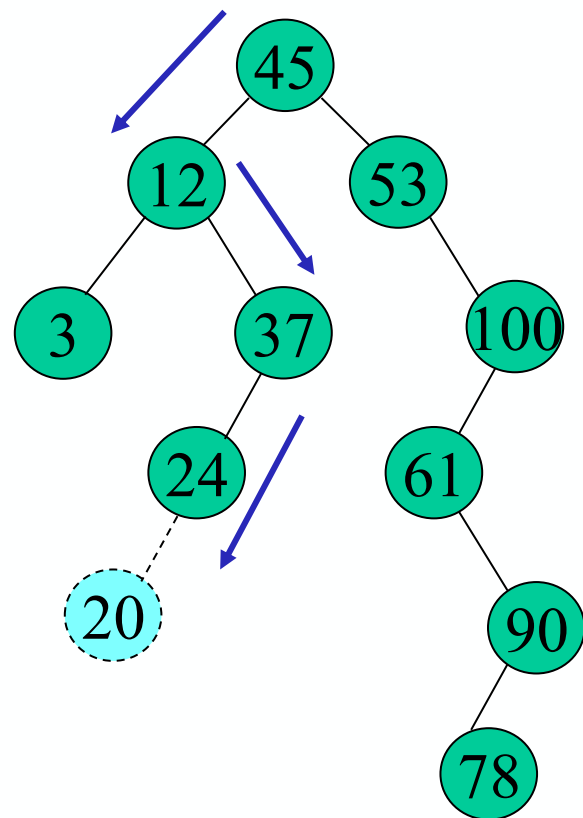
```
BSTree SearchBST(BSTree T, KeyType key) {  
    /*若二叉排序树为空, 则查找失败, 返回空指针T;  
    若key等于T→data.key, 则查找成功, 返回根结点地址*/  
    if((!T) || key==T→data.key)  
        return T;  
    //若key小于T→data.key, 则进一步查找左子树  
    else if (key<T→data.key)  
        return SearchBST(T→lchild,key);  
    //若key大于T→data.key, 则进一步查找右子树  
    else  
        return SearchBST(T→rchild,key);  
}
```

(2) 二叉排序树的操作 - 插入(P239)

要将一个关键字值为k的新结点插入到二叉排序树中，需要从**根结点**向下查找当树中不存在关键字等于k的结点，才进行插入，且**一定插入在叶结点上**

插入算法过程描述：

- (1) 若二叉排序树T为空，则创建一个key域为k的结点，将它作为根结点；
- (2) 否则将k和根结点的关键字 $T \rightarrow \text{key}$ 比较，若两者相等，则说明树中已有此关键字k，无须插入，直接返回0；
- (3) 若 $k < T \rightarrow \text{key}$ ，则将k插入根结点的左子树中。
- (4) 否则将它插入右子树中。



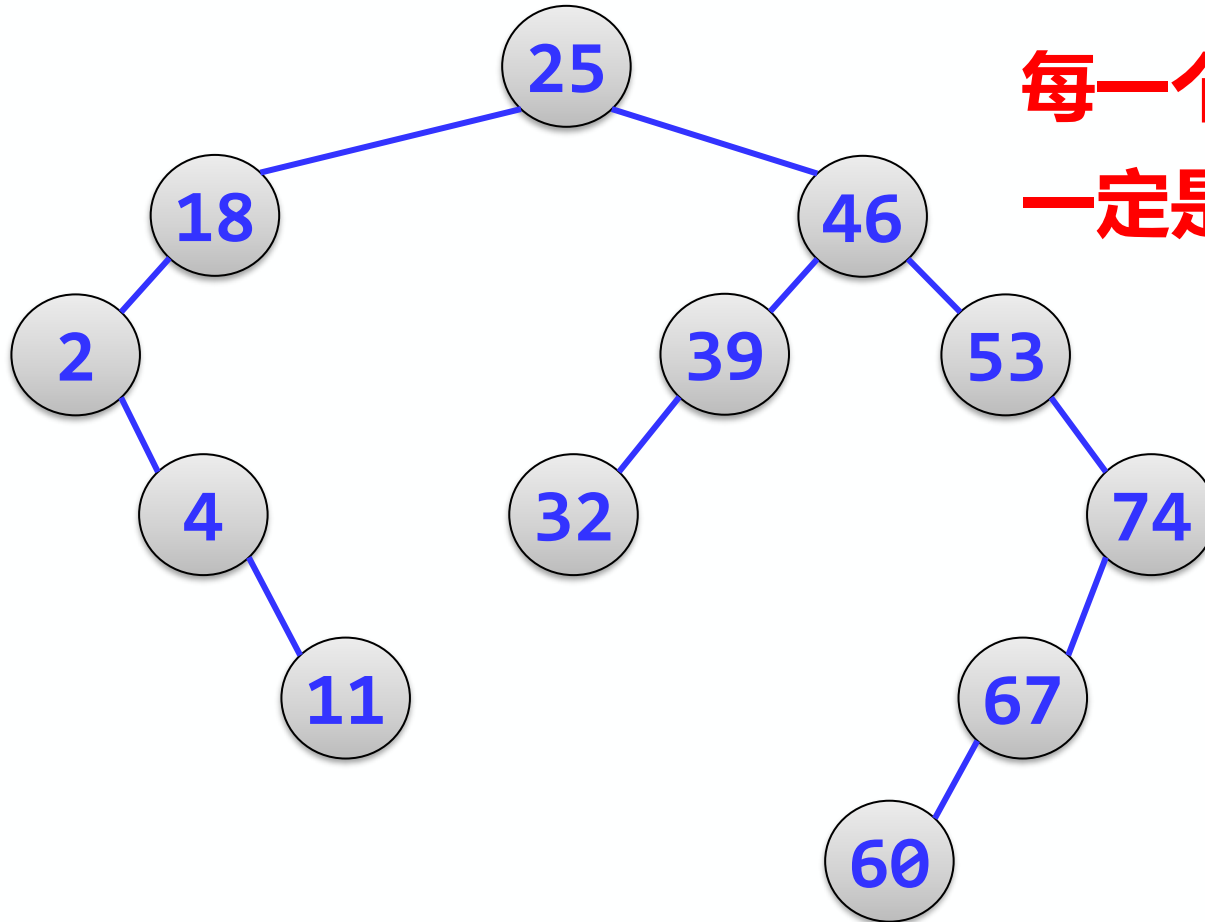
插入结点20

【二叉排序树插入算法描述】 (P240)

```
BSTree InsertBST(BSTree &T, KeyType key) {  
    /*若二叉排序树T中不存在关键字等于key的数据元素, 则插入*/  
    if(!T) {  
        S=new BSTNode; //生成新结点S  
        S→data.key=key; //新结点S的关键字为key  
        S→lchild=S→rchild=NULL; //待插入的新结点一定是叶子结点  
    }  
    //若key小于T->data.key, 则将该关键字插入到左子树  
    else if (key<T→data.key)  
        InsertBST(T→lchild,key);  
    //若key大于T->data.key, 则将该关键字插入到右子树  
    else  
        InsertBST(T→rchild,key);  
}
```

(2) 二叉排序树的操作 - 插入 (创建) 240

序列: 25 18 46 2 53 39 25 32 4 74 67 60 11



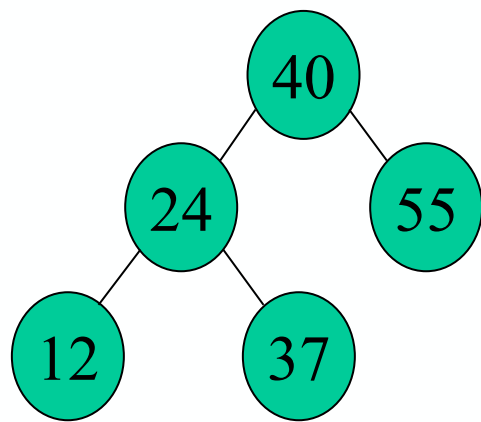
每一个新插入元素
一定是叶结点

二叉排序树创建完毕

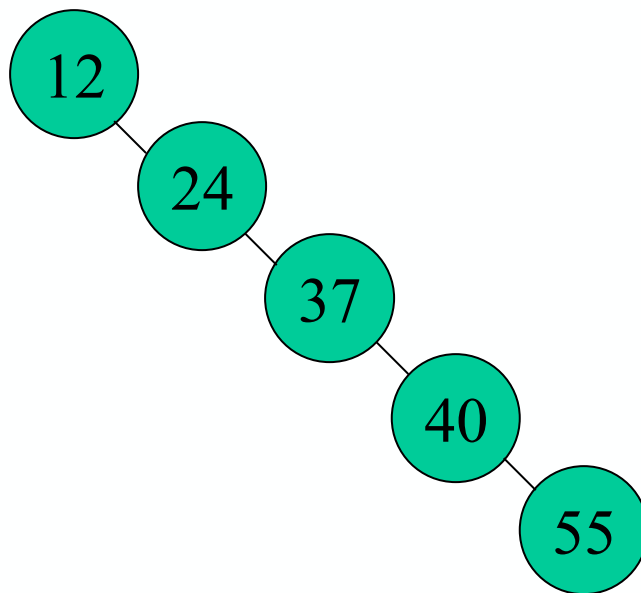
(2) 二叉排序树的操作 - 插入 (P241)

不同插入次序的相同数据，生成不同形态的二叉排序树

40, 24, 12, 37, 55



12, 24, 37, 40, 55



$$ASL = \frac{1}{5} (1 + 2 + 2 + 3 + 3) = \frac{11}{5}$$

$$ASL = \frac{1}{5} (1 + 2 + 3 + 4 + 5) = \frac{15}{5}$$

二叉排序树的高度越高，ASL越大

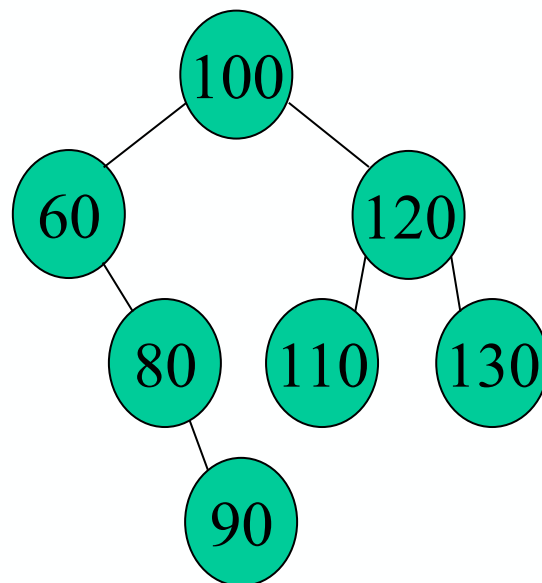
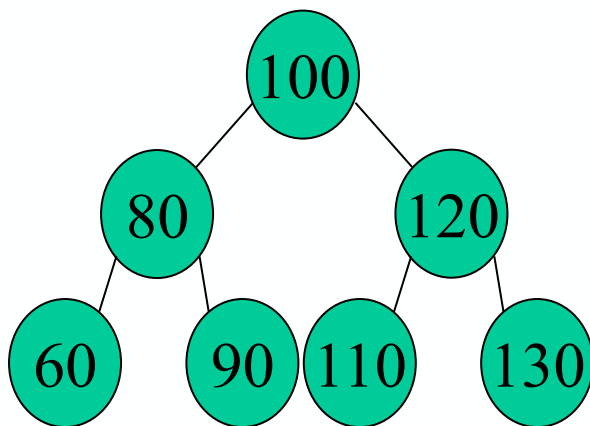
【练习4】 分别以下列序列构造二叉排序树，与用其它三个序列所构造的结果不同的是（ ）。

A. (100, 80, 90, 60, 120, 110, 130)

B. (100, 120, 110, 130, 80, 60, 90)

C. (100, 60, 80, 90, 120, 110, 130)

D. (100, 80, 60, 90, 120, 130, 110)

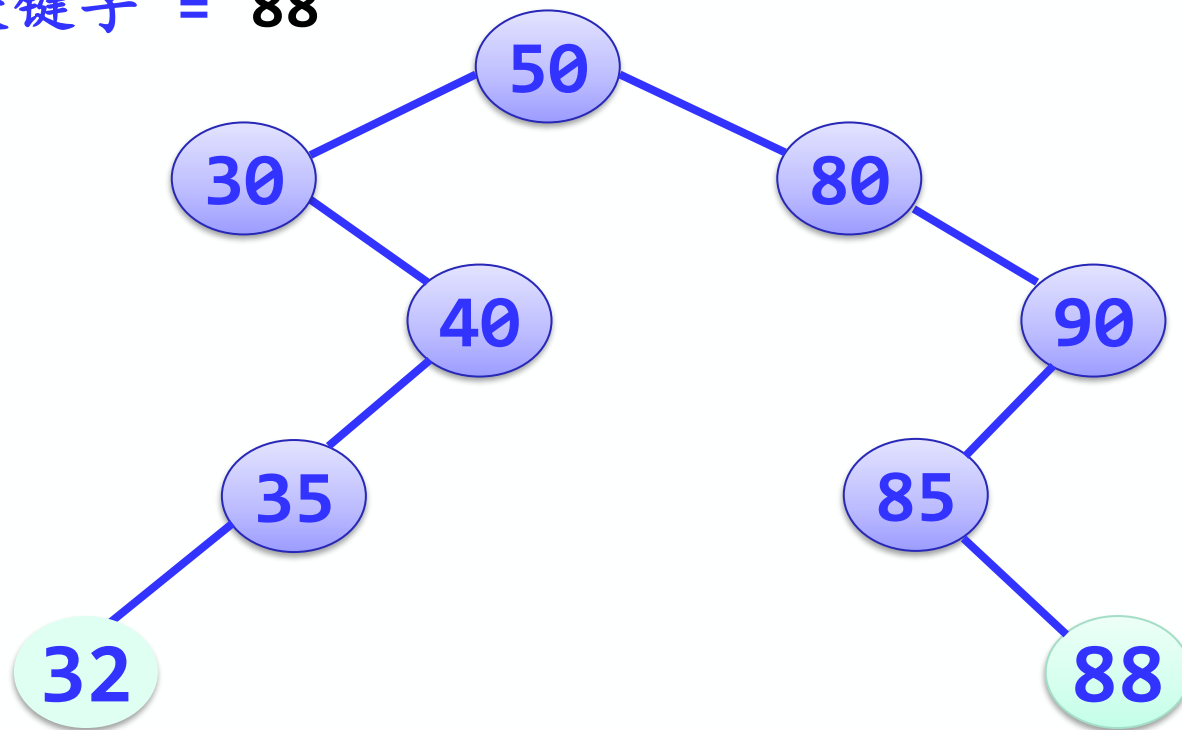


答案： C

(3) 二叉排序树的操作 - 删除(P241)

(1) 被删除的结点是叶子结点：直接删去该结点。删除之后不会破坏整棵树的结构(仍然符合二叉排序树的要求)

被删关键字 = 88

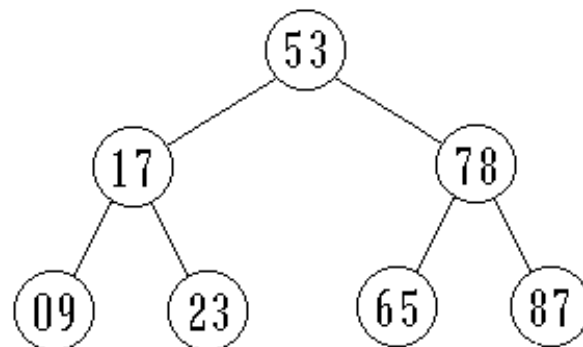
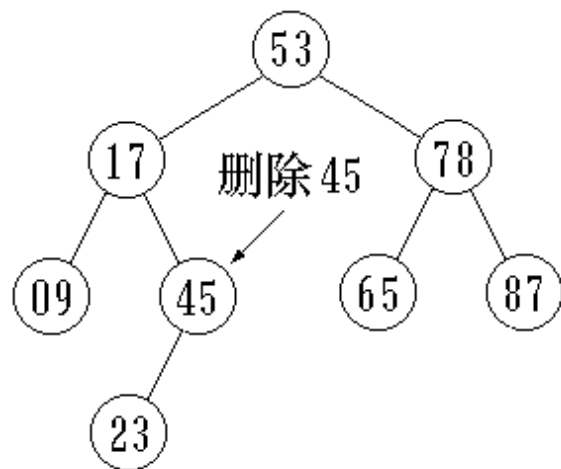


中序遍历: 30 32 35 40 50 80 85 88 90

(3) 二叉排序树的操作 - 删除

(2) 被删除的结点只有左子树或者只有右子树，用其左子树或者右子树替换它（结点替换）。

只有左子树



缺右子树用左子女填补

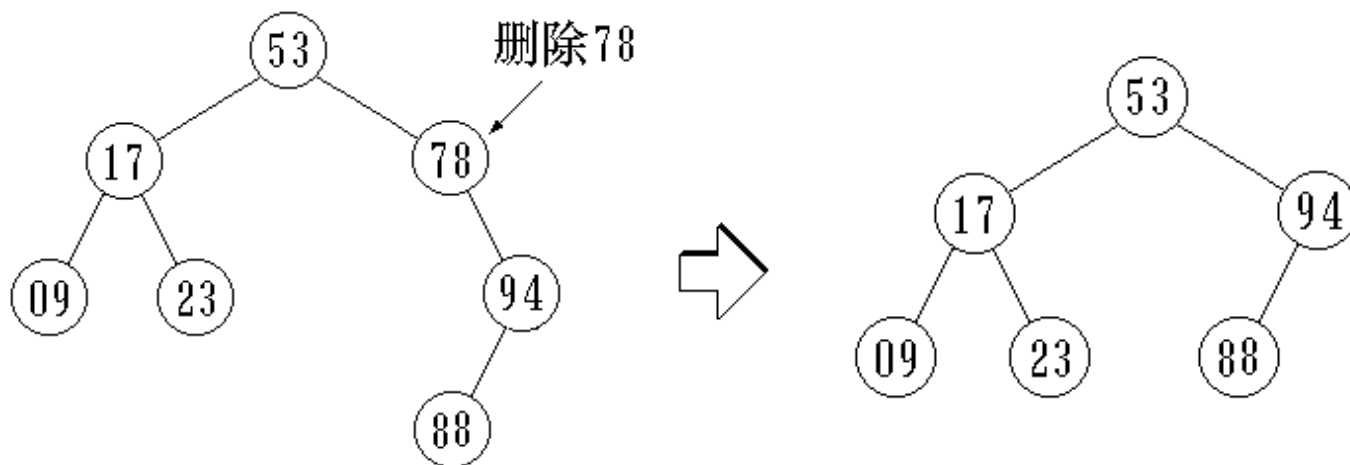
中序遍历: 09 17 23 45 53 65 78 87

被删除结点45所处的位置: 作为17右子树的一部分 $45 > 17$
作为53左子树的一部分 $45 < 53$

(3) 二叉排序树的操作 - 删除

(2) 被删除的结点只有左子树或者只有右子树，用其左子树或者右子树替换它（结点替换）。

只有右子树



缺左子树用右子女填补

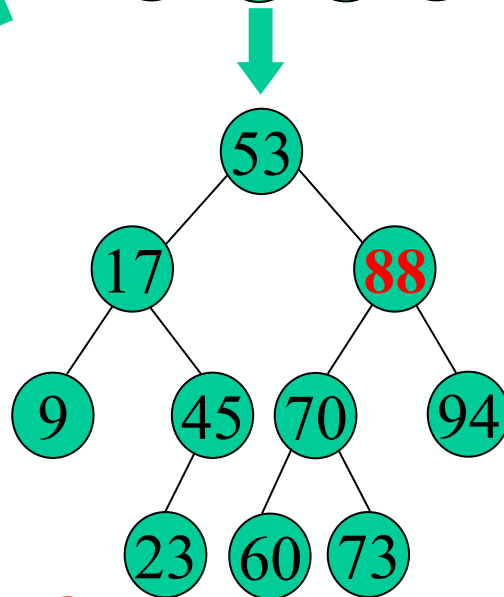
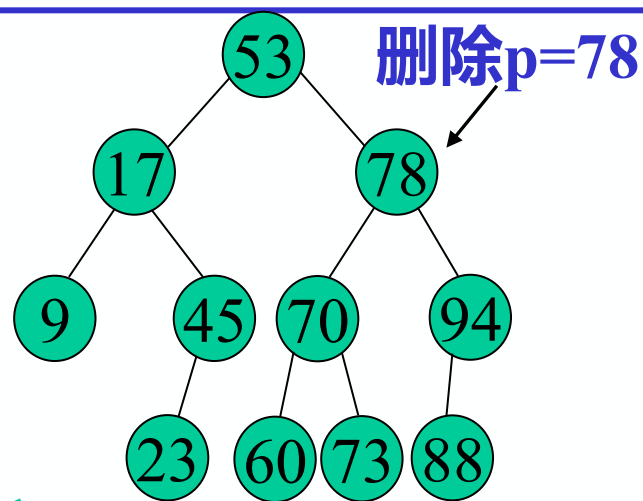
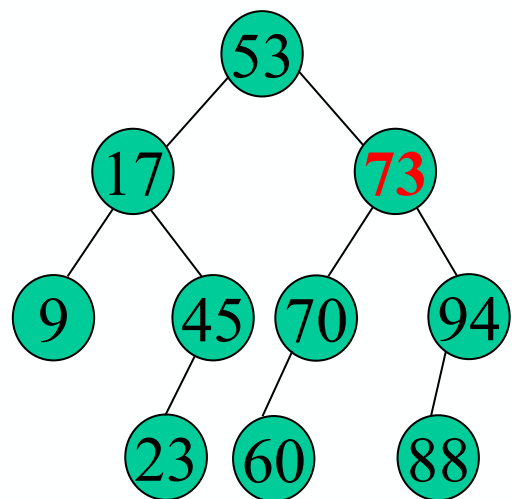
被删除结点78所处的位置：作为53右子树的一部分

(3) 二叉排序树的操作 - 删除

(3) 若被删结点p既有左子树又有右子树，任选以下两种处理方法后，转为前两种删除情况：

- ◆ 即从p的左子树中选择值最大的结点s放在p的位置，并删除s。

- ◆ 即从p的右子树中选择值最小的结点s放在p的位置，并删除s。（P241）



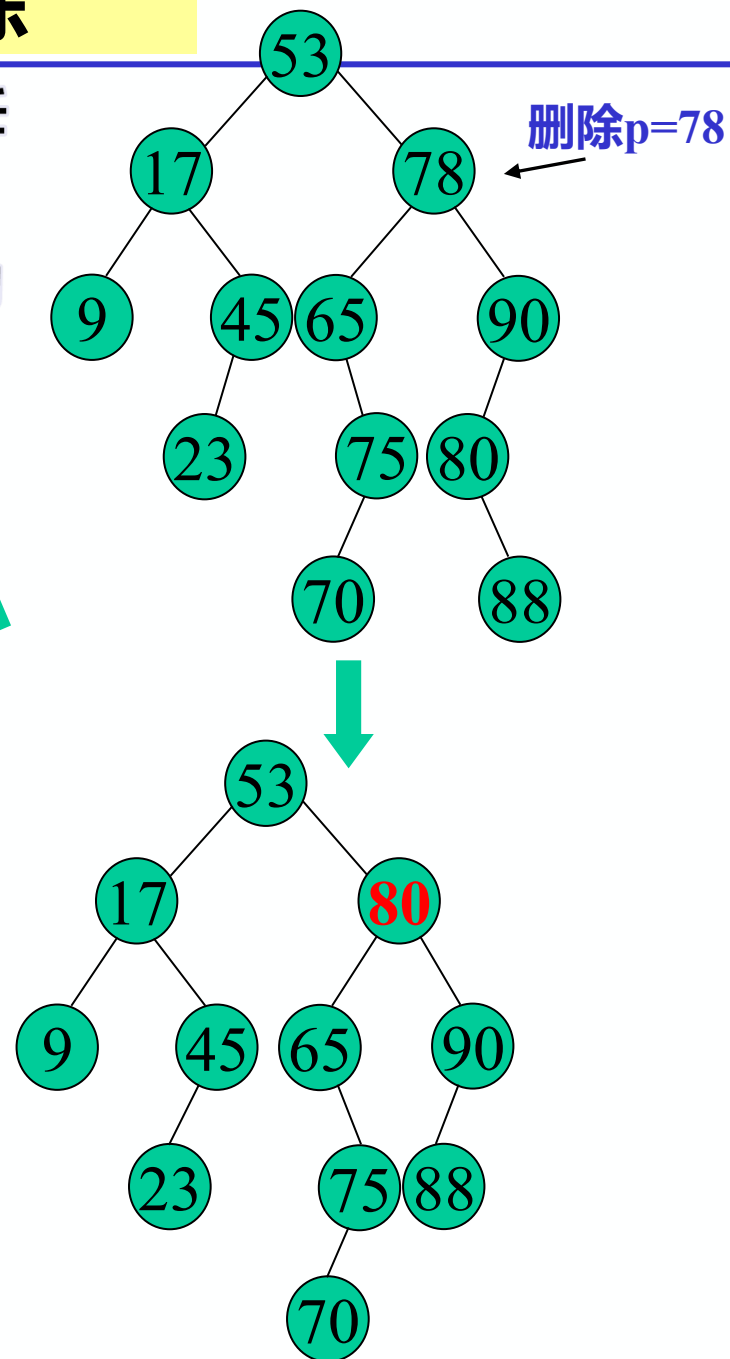
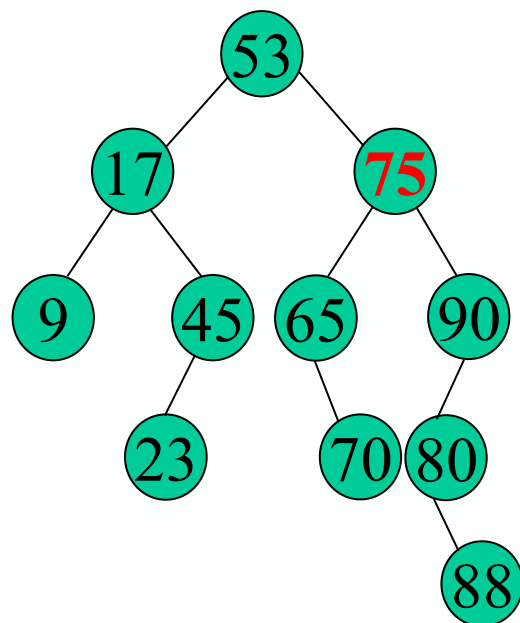
中序遍历: 9 17 23 45 53 60 70 73 78 88 94

(3) 二叉排序树的操作 - 删除

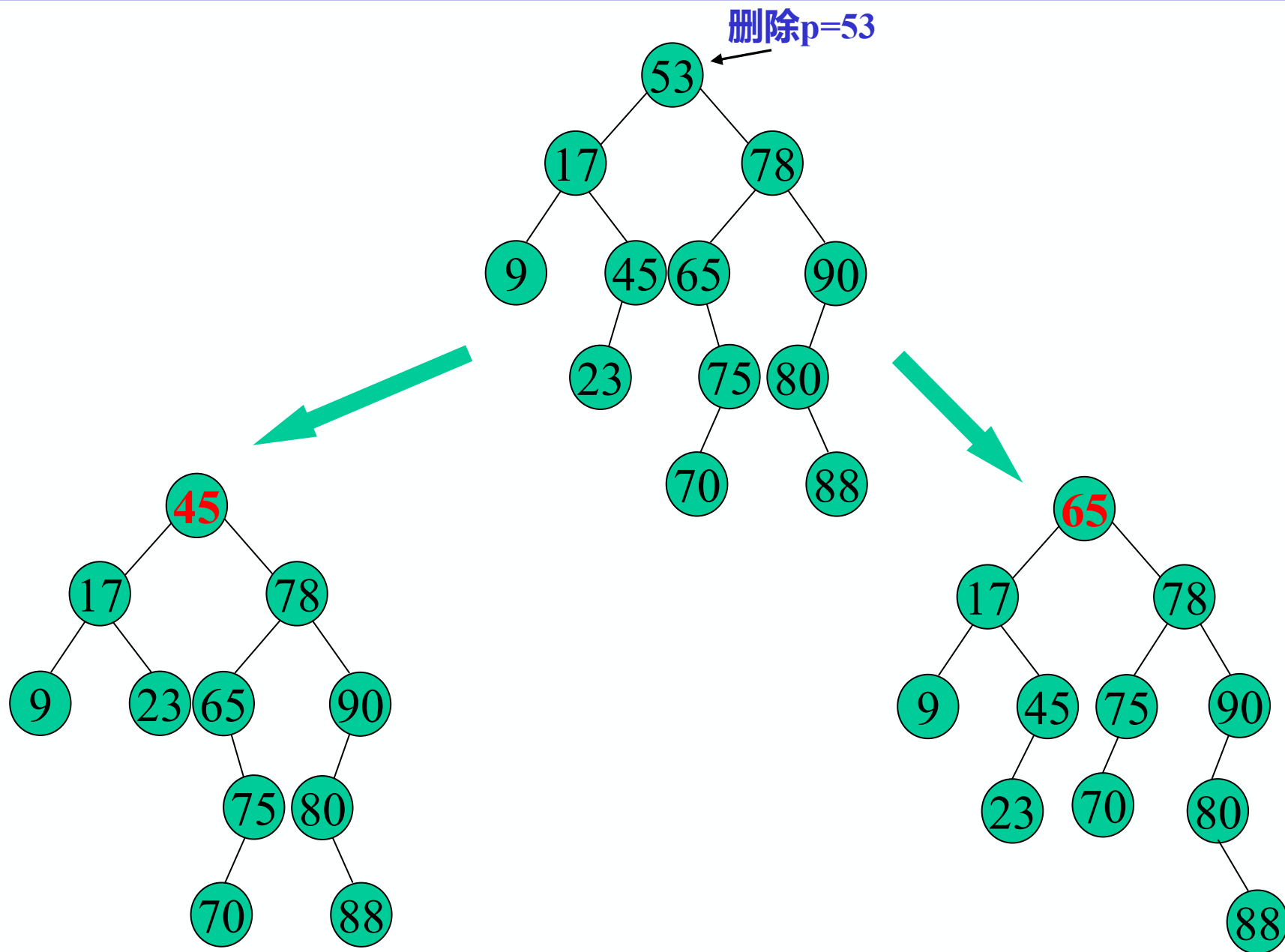
(3) 若被删结点p既有左子树又有右子树，任选以下两种处理方法后，转为前两种删除情况：

- ◆ 即从p的左子树中选择值最大的结点s放在p的位置，并删除s。

- ◆ 即从p的右子树中选择值最小的结点s放在p的位置，并删除s。



(3) 二叉排序树的删除



(3) 二叉排序树的操作 - 删除

二叉排序树的删除总结：

- 如果被删除结点 p 是叶结点，则直接删除；
- 若结点 p 只有一棵左子树或右子树，则让 p 的左子树/右子树替代 p 的位置，成为 p 双亲结点的子树；
- 若结点 p 有左右两棵子树，从 p 的左子树中选择值最大的结点 s 放在 p 的位置，或从 p 的右子树中选择值最小的结点 t 放在 p 的位置。

{令 p 中序序列中直接前驱（或直接后继）替代 p ，然后从二叉排序树中删去这个直接前驱（或直接后继），然后转为前两种情况。}

【练习5】①按（）遍历二叉排序树得到的序列是一个有序序列；

②在二叉排序树中进行查找的效率与（）有关；

③在常用的描述二叉排序树的存储结构中，关键字值最大的结点（）。

①A. 先序 B. 中序 C. 后序 D. 层序

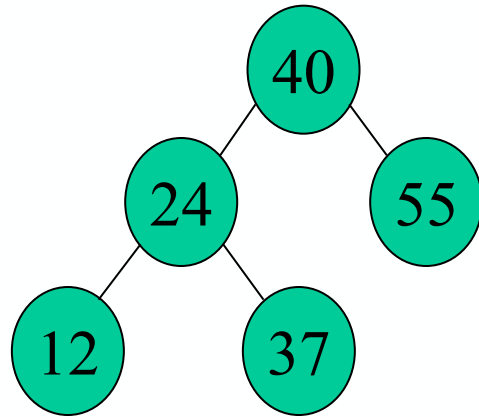
**②A. 二叉排序树结点的个数 B. 被查找结点的度
C. 二叉排序树的深度 D. 二叉排序树的存储结构**

**③A. 左指针一定为空 B. 右指针一定为空
C. 左右指针均为空 D. 左右指针均不为空**

答案：B C B

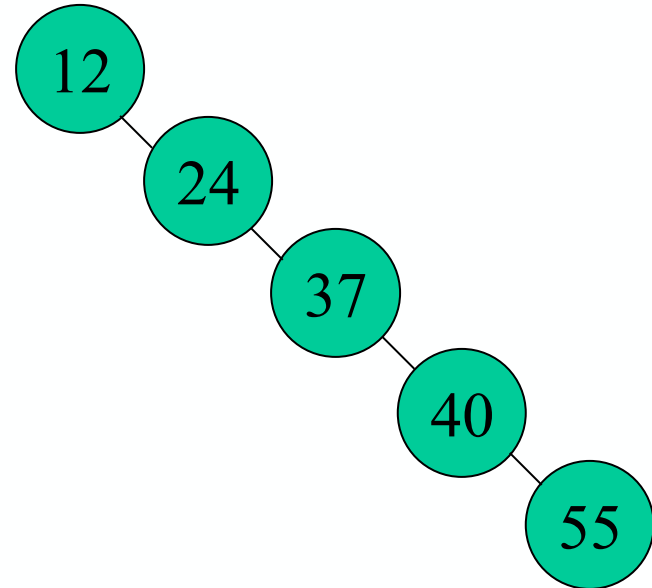
不同插入次序的相同数据，生成不同形态的二叉排序树

40, 24, 12, 37, 55



$$ASL = \frac{1}{5} (1 + 2 + 2 + 3 + 3) = \frac{11}{5}$$

12, 24, 37, 40, 55



$$ASL = \frac{1}{5} (1 + 2 + 3 + 4 + 5) = \frac{15}{5}$$

二叉排序树的高度越高，ASL越大，查找效率越差

7.3.2 平衡二叉树(P244)

问题：如何提高二叉排序树的查找效率？

答：事实上，树的高度越小，ASL值越小，查找速度越快。

即尽量让二叉排序树的形状均衡。



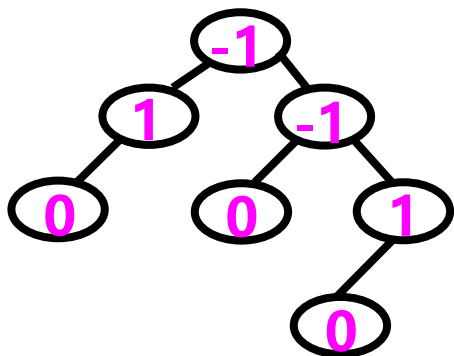
平衡二叉树

➤ 若一棵二叉排序树中，每个结点的左右子树的高度至多相差1，且左、右子树都是平衡二叉树，那么称此二叉排序树为平衡二叉树(AVL树)。

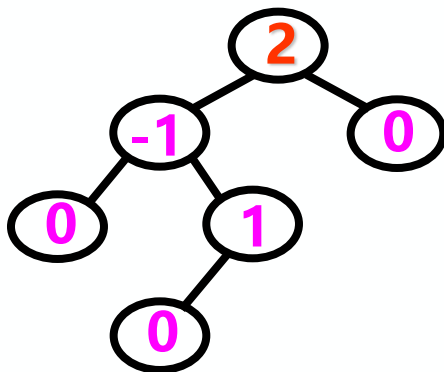
平衡因子：结点的左子树高度减去右子树高度。

平衡二叉树中任一结点的平衡因子只能取：**-1**、**0** 或 **1**，
否则这棵二叉树就失去平衡，不再是平衡二叉树。

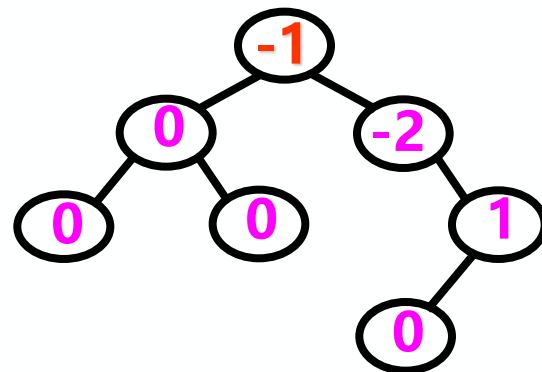
练习：判断下列二叉树是否平衡二叉树？



(a) 平衡二叉树



(b) 不平衡树



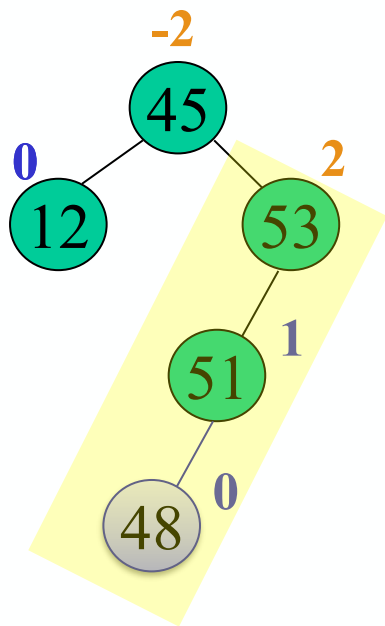
(c) 不平衡树

叶子结点的平衡因子一定为0

在AVL中，每个结点平衡因子的绝对值 ≤ 1

平衡二叉树(P244)

插入48



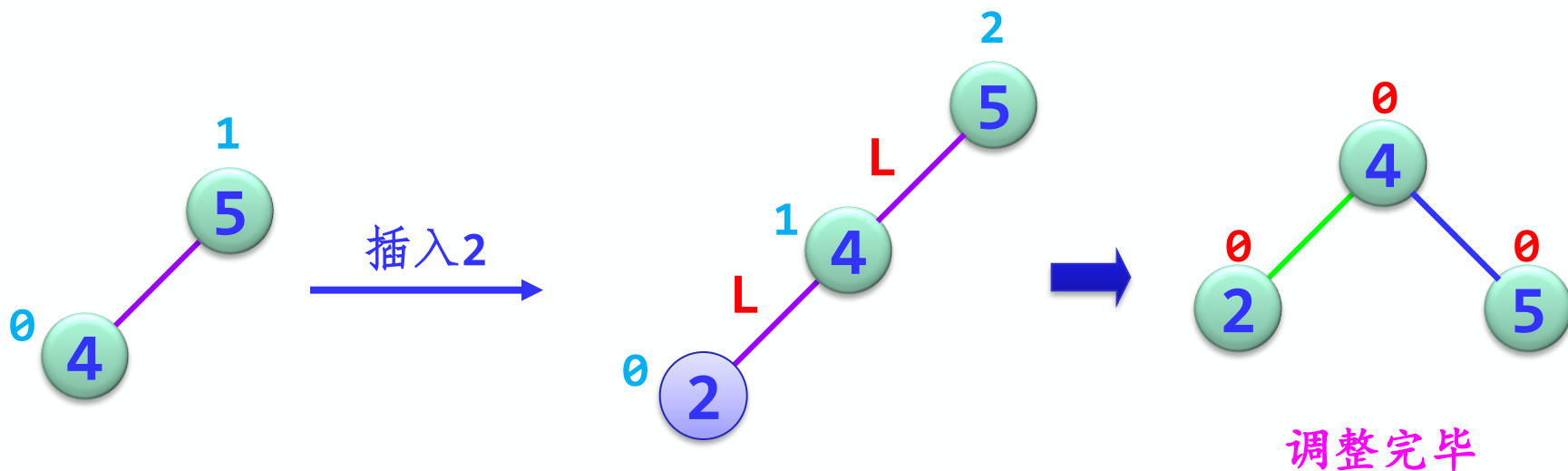
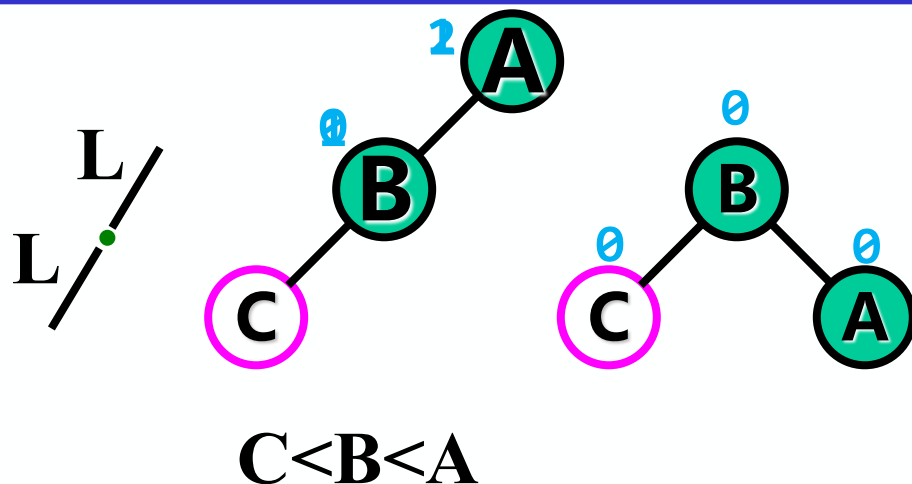
- 如果在一棵平衡二叉树中插入一个新结点，就有可能造成失衡，此时必须**重新调整树的结构**，使之恢复平衡。我们称调整平衡过程为**平衡旋转**。
- 当不止一个失衡结点时，**调整最小失衡子树**，即在插入路径上，离插入结点最近的、平衡因子绝对值大于1的结点作为根结点的子树（一共包含三个结点）。

- ✓ LL平衡旋转
- ✓ RR平衡旋转
- ✓ LR平衡旋转
- ✓ RL平衡旋转

保证二叉排序树的性质不变

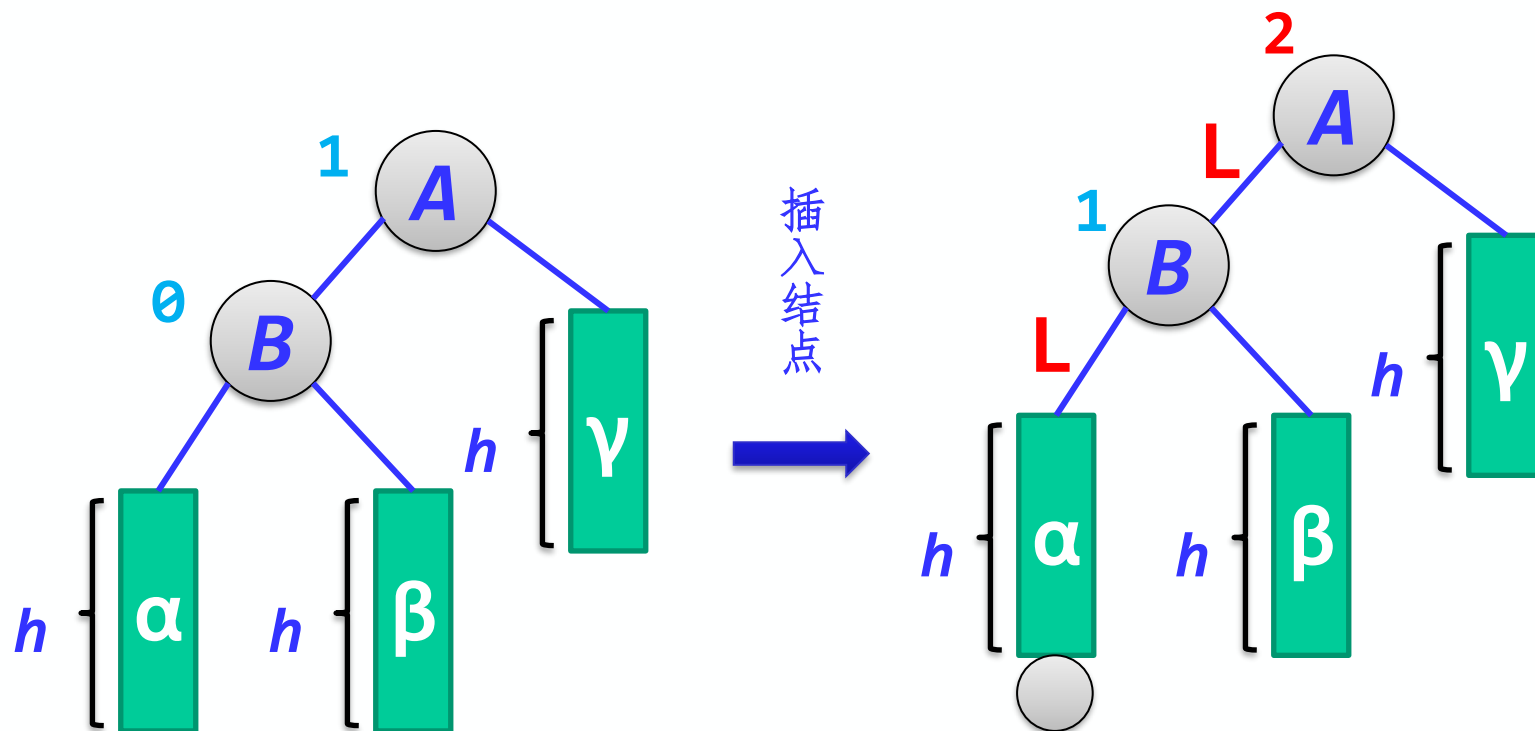
1) LL平衡旋转:

若在A的左子树的左子树上插入新结点C, 使A的平衡因子从1增至2, 需要将失衡结点顺时针旋转90°。

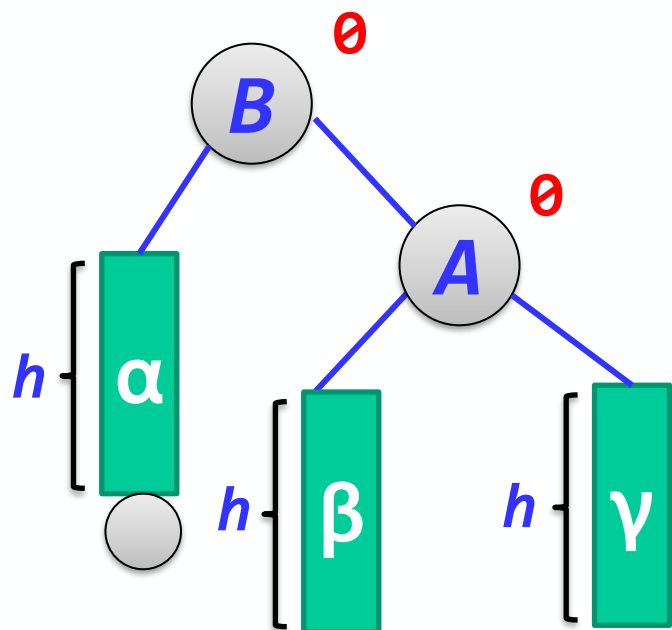


插入关键字2的结果

1) LL平衡旋转:



1) LL平衡旋转:



LL调整后的结果

LL型调整过程:

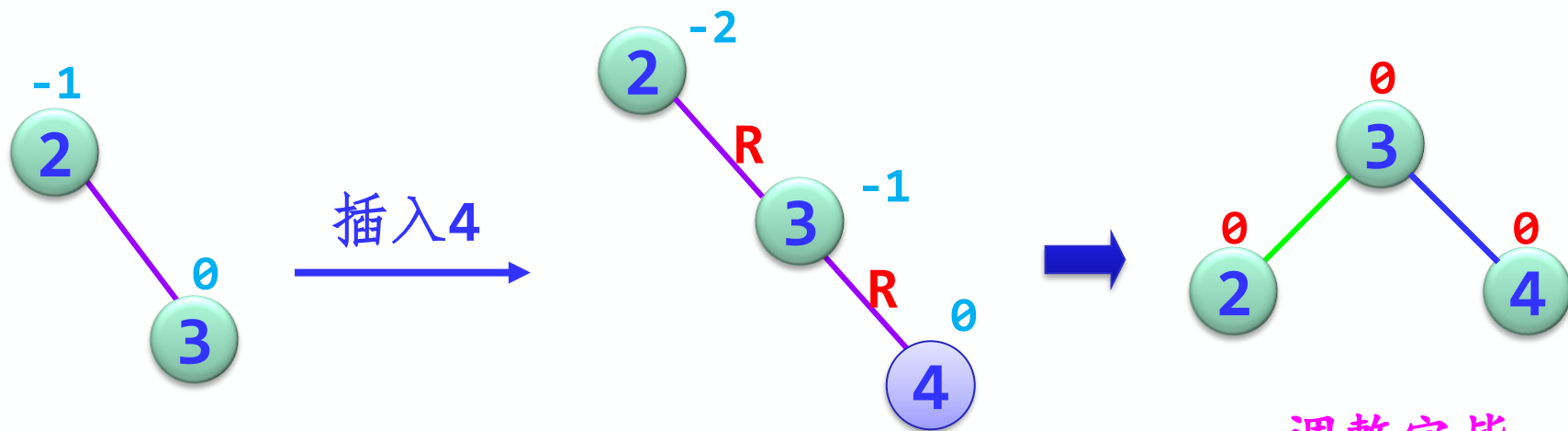
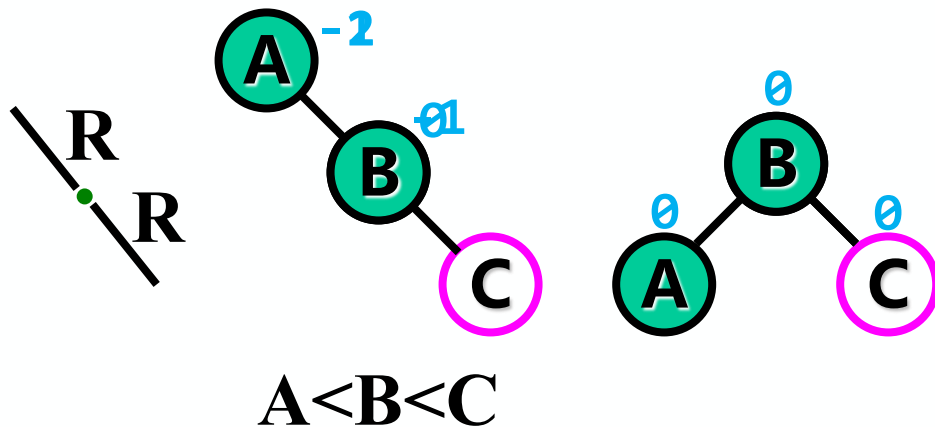
- B结点带左子树 α 一起上升
- A结点成为B的右孩子
- 原来B结点的右子树 β 作为A的左子树

新插入的结点 $< B < A$

$B < \beta < A$

2) RR平衡旋转:

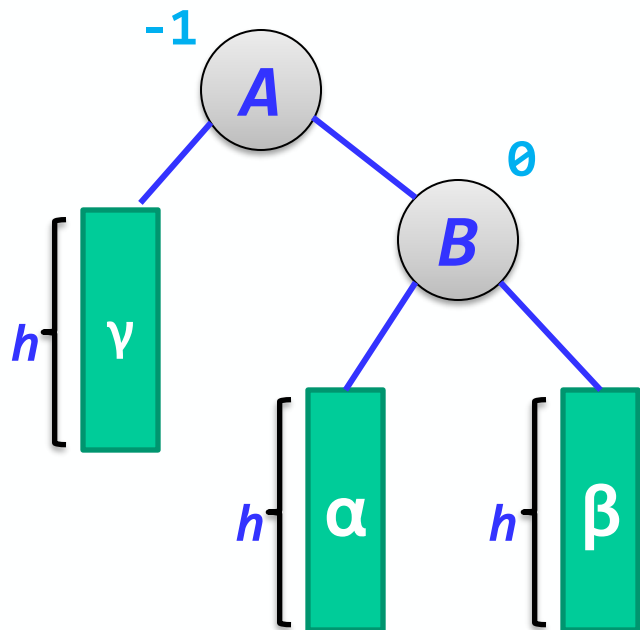
若在A的右子树的右子树上插入新结点C，使A的平衡因子从-1降至-2，需要将失衡结点逆时针旋转90°。



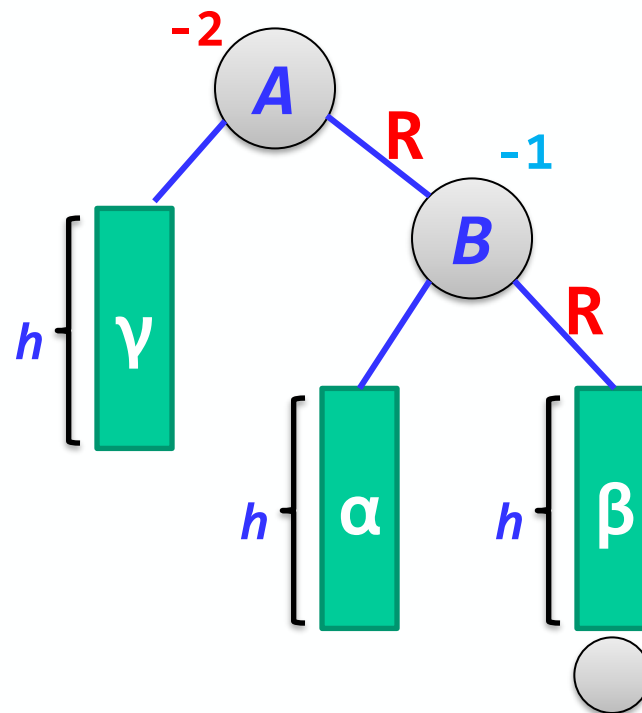
调整完毕

插入关键字4的结果

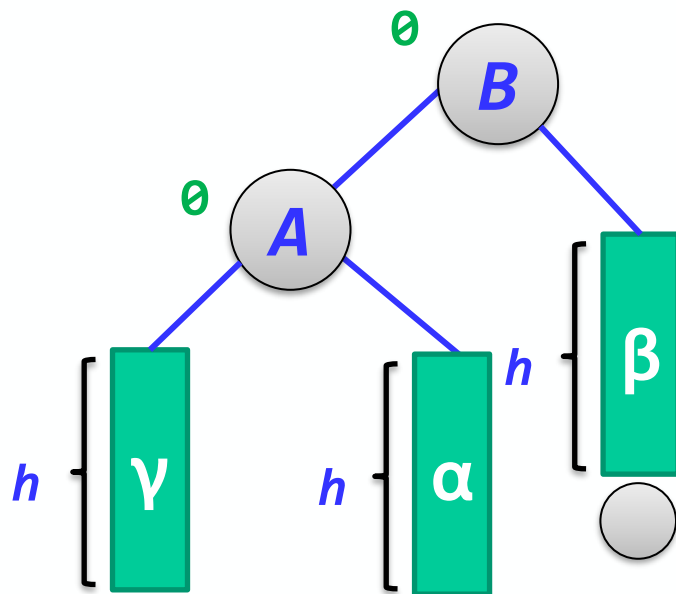
2) RR平衡旋转:



插入结点
→



2) RR平衡旋转:



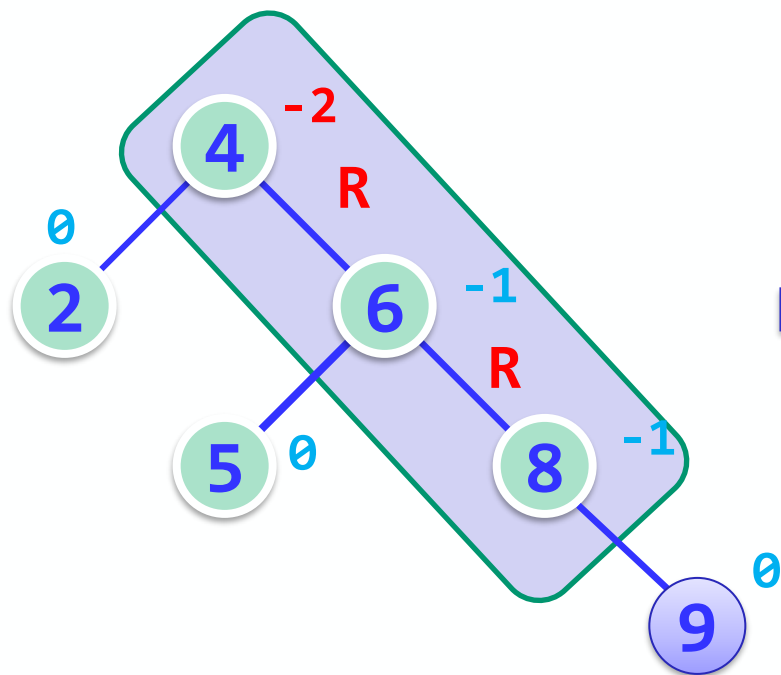
RR调整后的结果

RR型调整过程:

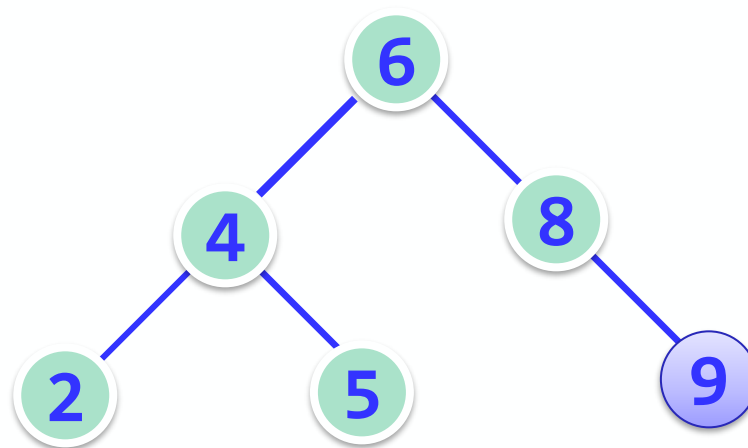
- **B**结点带右子树 β 一起上升
- **A**结点成为**B**的左孩子
- 原来**B**结点的左子树 α 作为**A**的右子树

A < B < 新插入的结点

2) RR平衡旋转:



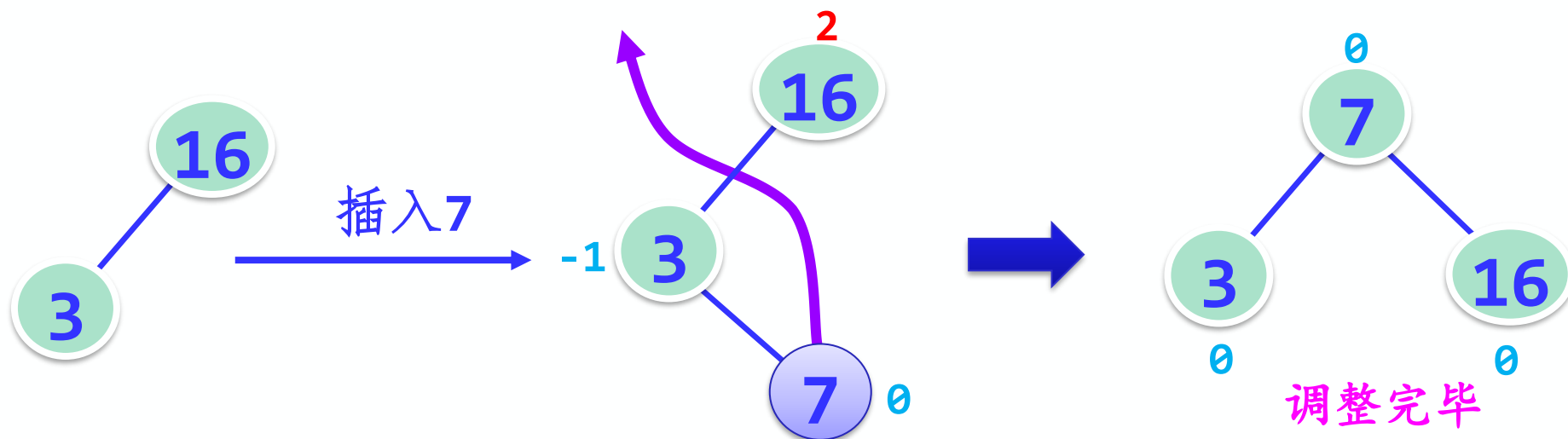
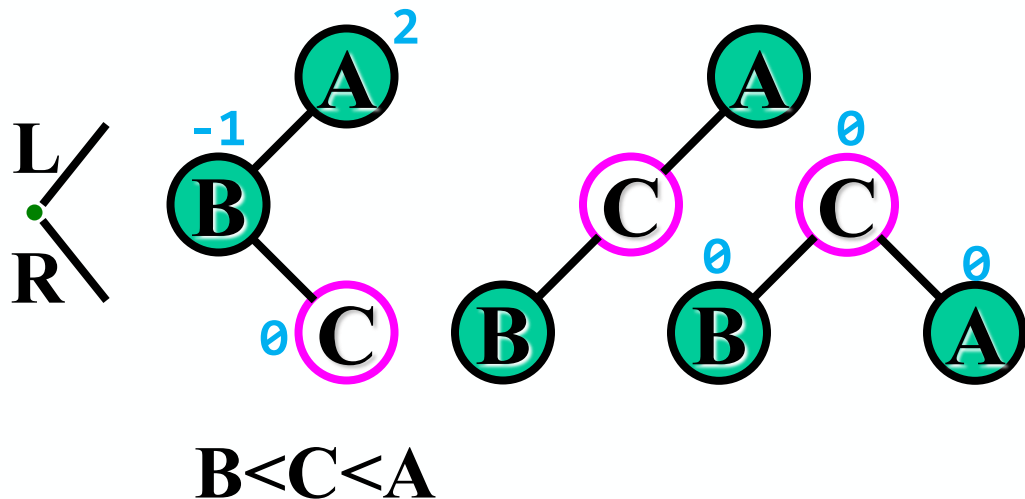
插入9



调整完毕

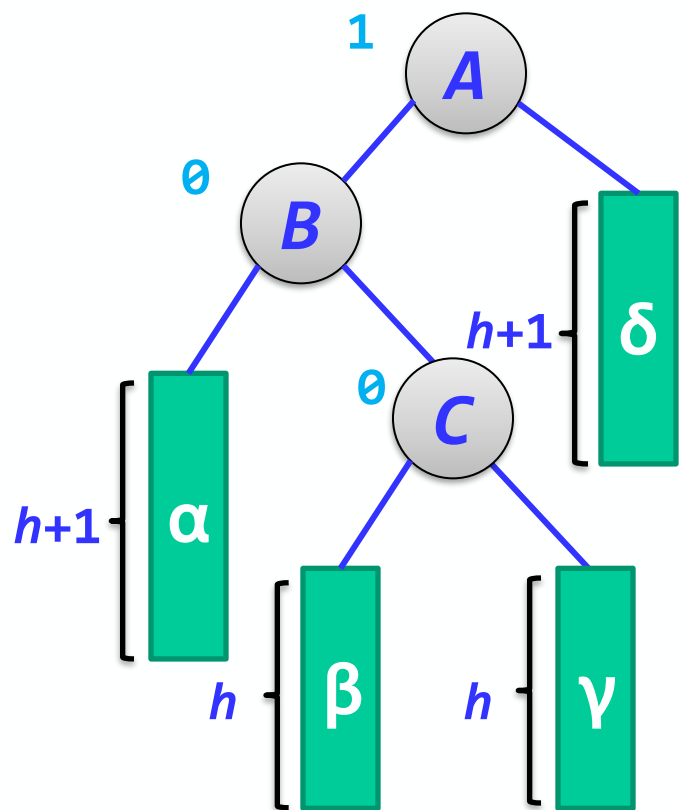
3) LR平衡旋转:

若在A的左子树的右子树上插入结点，使A的平衡因子从1增加至2，需要对B、C先进行逆时针旋转，再顺时针旋转。

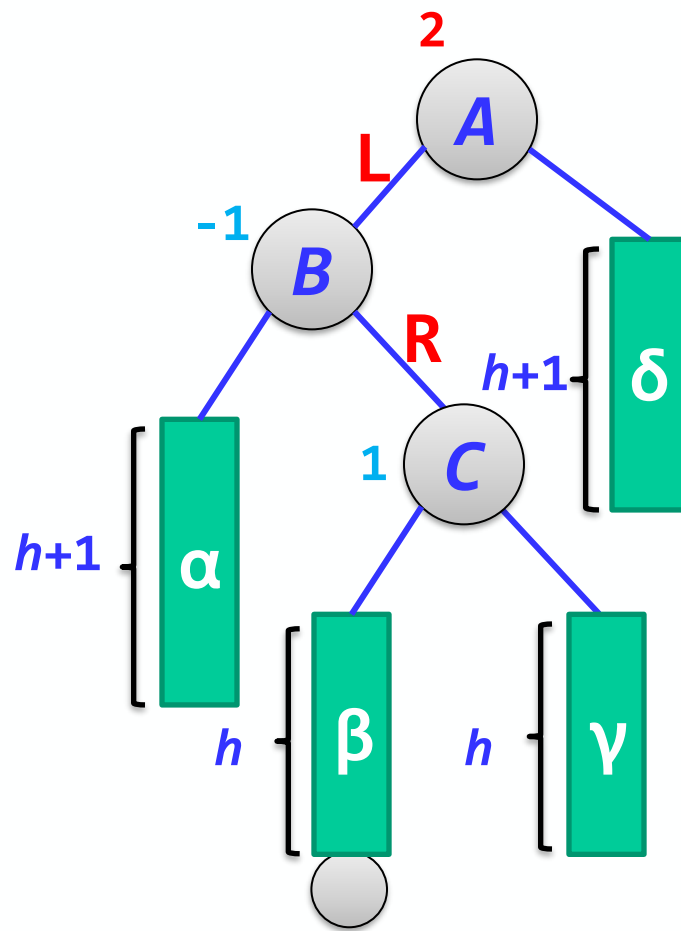


插入关键字7的结果

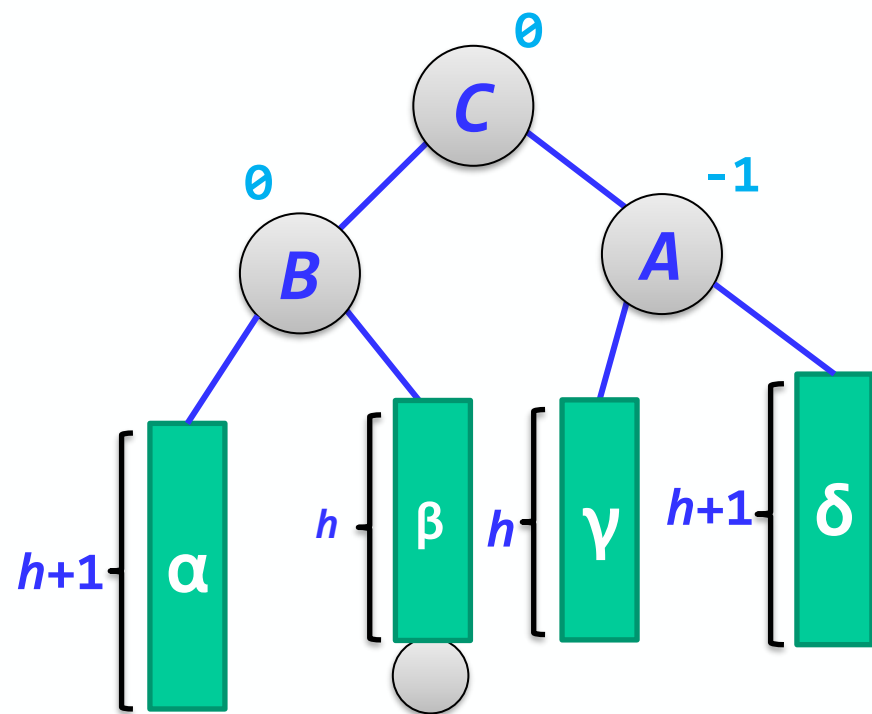
3) LR平衡旋转:



插入结点



3) LR平衡旋转:



LR调整后的结果

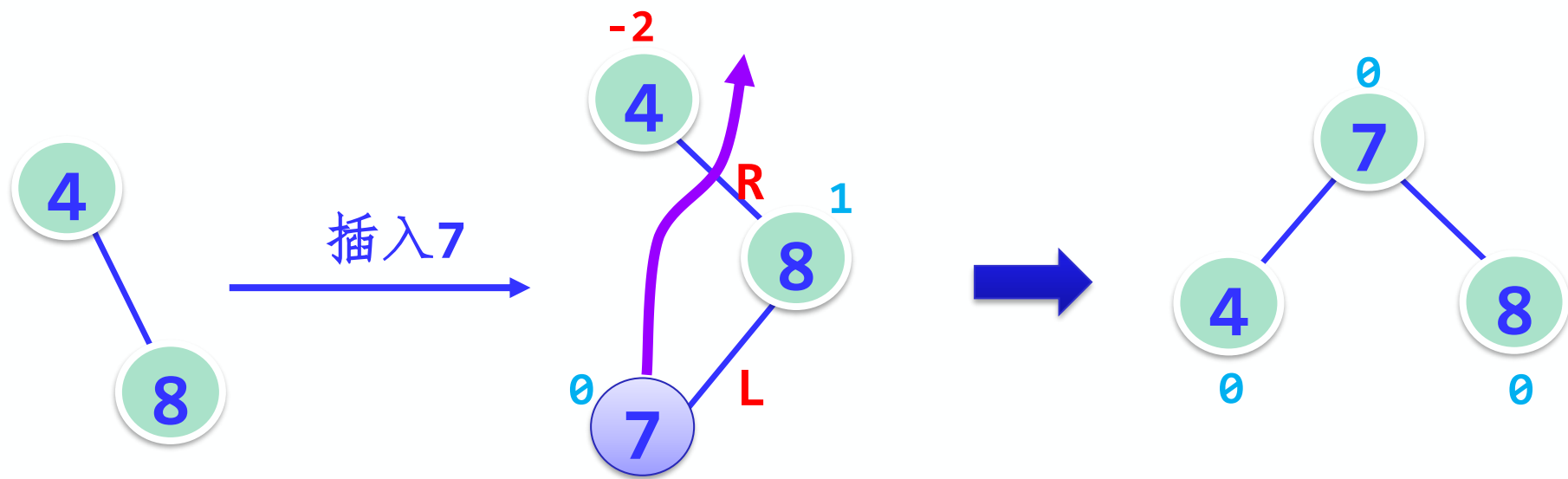
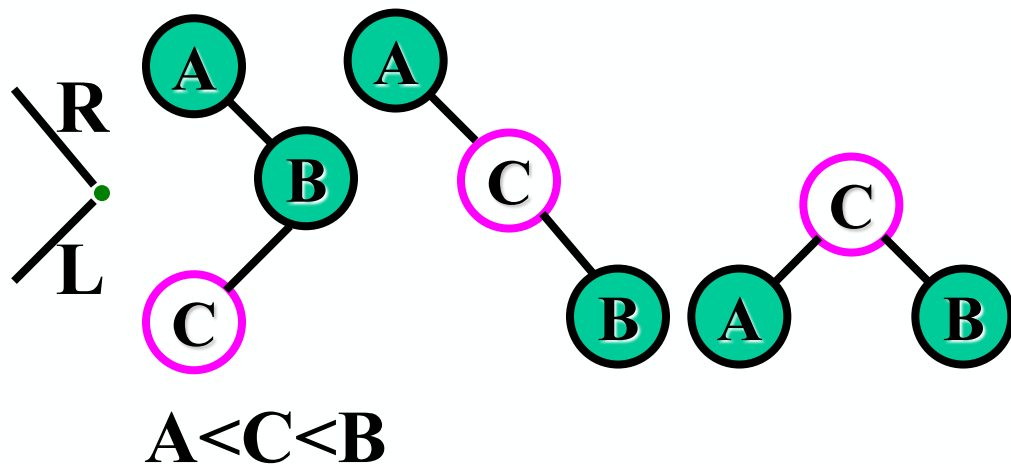
LR型调整过程:

- **C** 结点穿过 **A**、**B** 结点上升
- **B** 结点成为 **C** 的左孩子，**A** 结点成为 **C** 的右孩子
- 原来 **C** 结点的左子树 β 作为 **B** 的右子树；原来 **C** 结点的右子树 γ 作为 **A** 的左子树

$B < C < A$ ，中间值 **C 先穿越 **B** 和 **A** 上升成为根结点，**C** 的左右孩子分别给 **B** 和 **A**，作为它们的右和左孩子。**

4) RL平衡旋转:

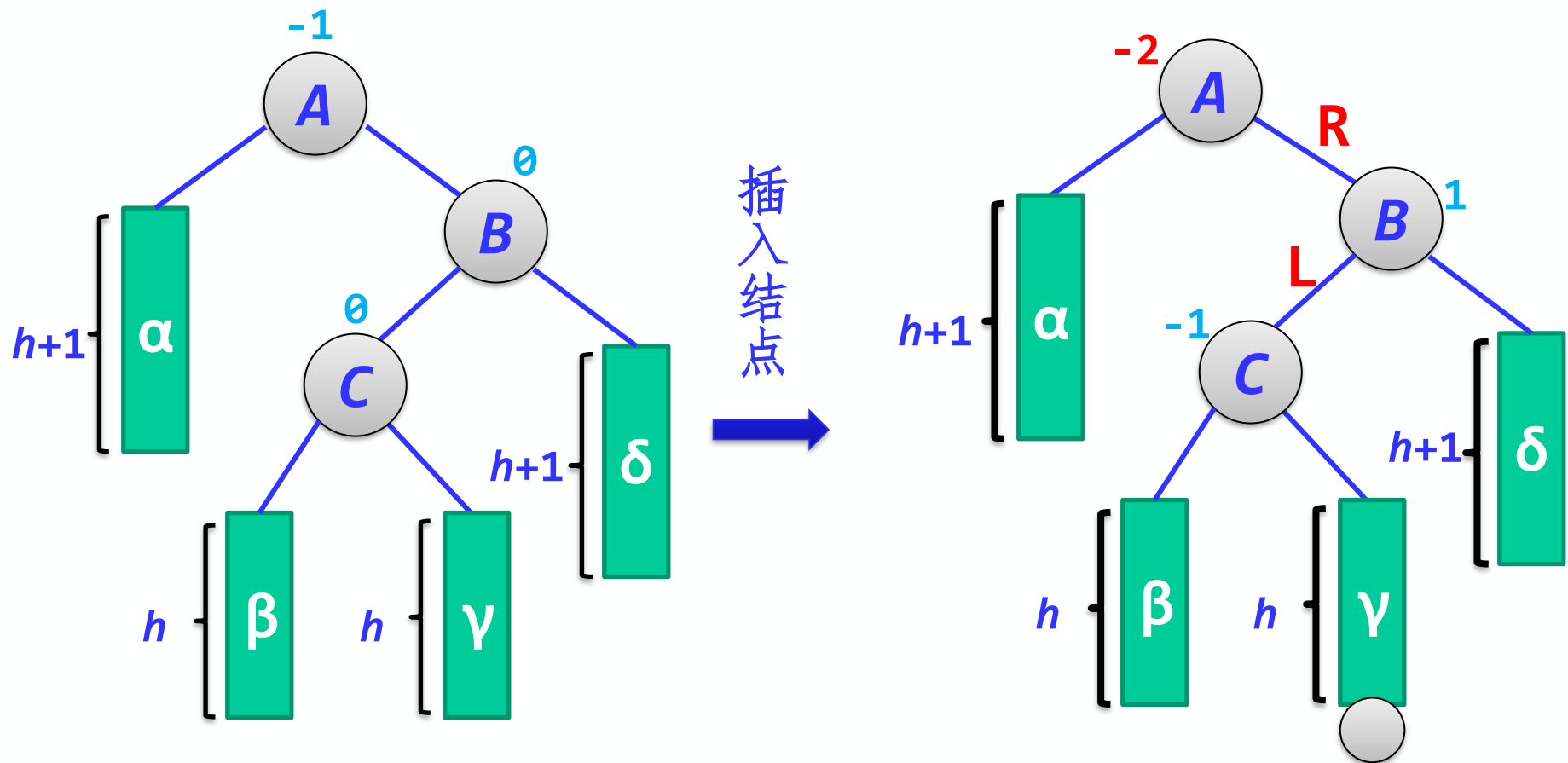
若在A的右子树的左子树上插入结点，使A的平衡因子从-1增加至-2，需要对B、C先进行顺时针旋转，再逆时针旋转。



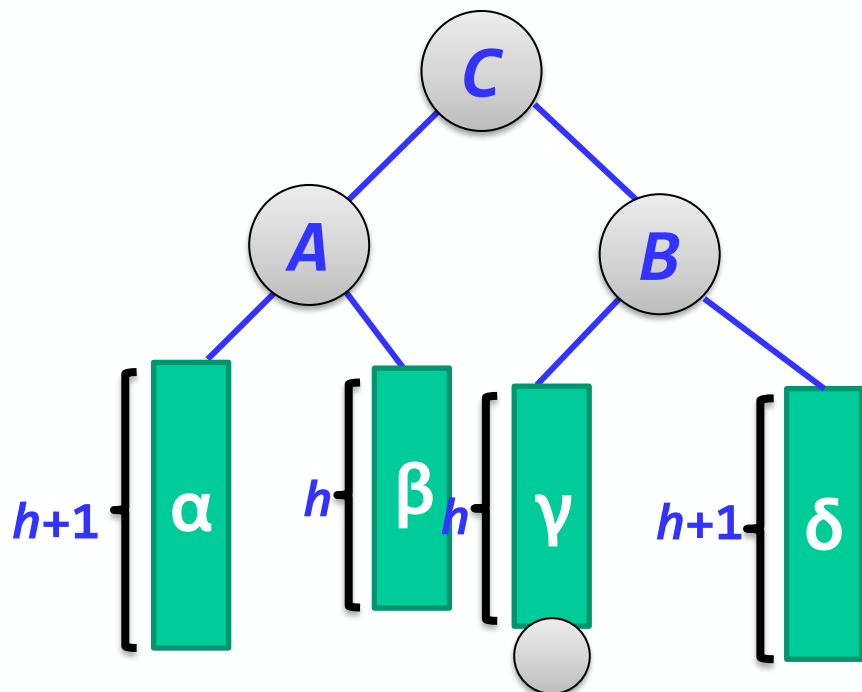
插入关键字7的结果

调整完毕

4) RL平衡旋转:



4) RL平衡旋转:

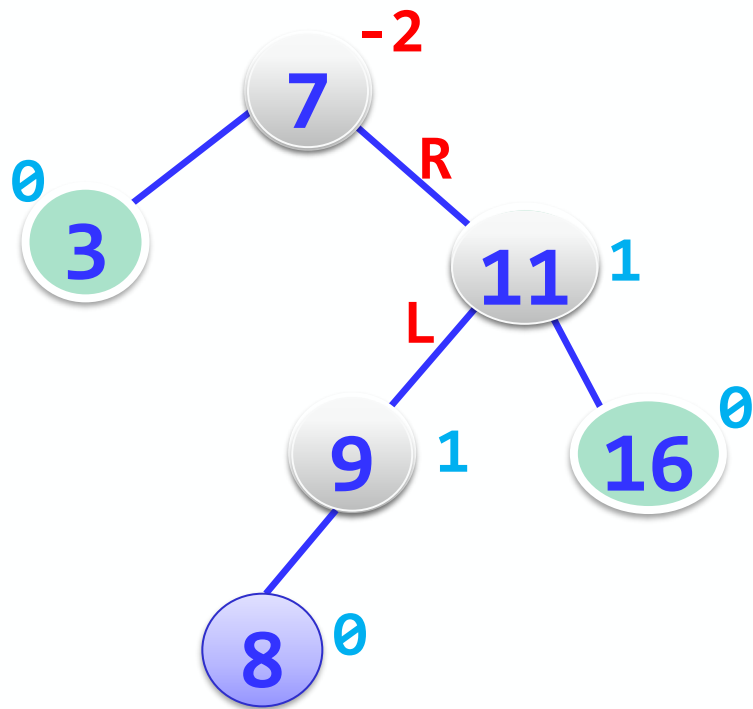


RL型调整过程:

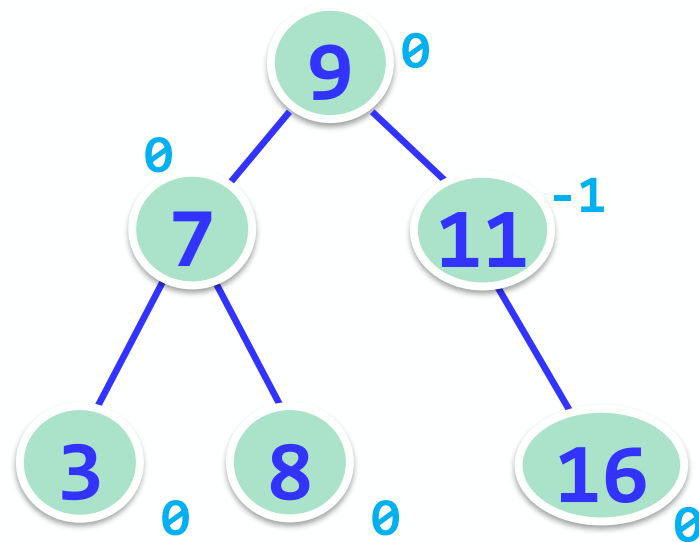
- **C** 结点穿过 **A**、**B** 结点上升
- **A** 结点成为 **C** 的左孩子，**B** 结点成为 **C** 的右孩子
- 原来 **C** 结点的左子树 **β** 作为 **A** 的右子树；原来 **C** 结点的右子树 **γ** 作为 **B** 的左子树

$A < C < B$ ，中间值 **C 先穿越 **AB** 上升成为根结点，**C** 的左右孩子分别给 **A** 和 **B**，作为它们的右和左孩子。**

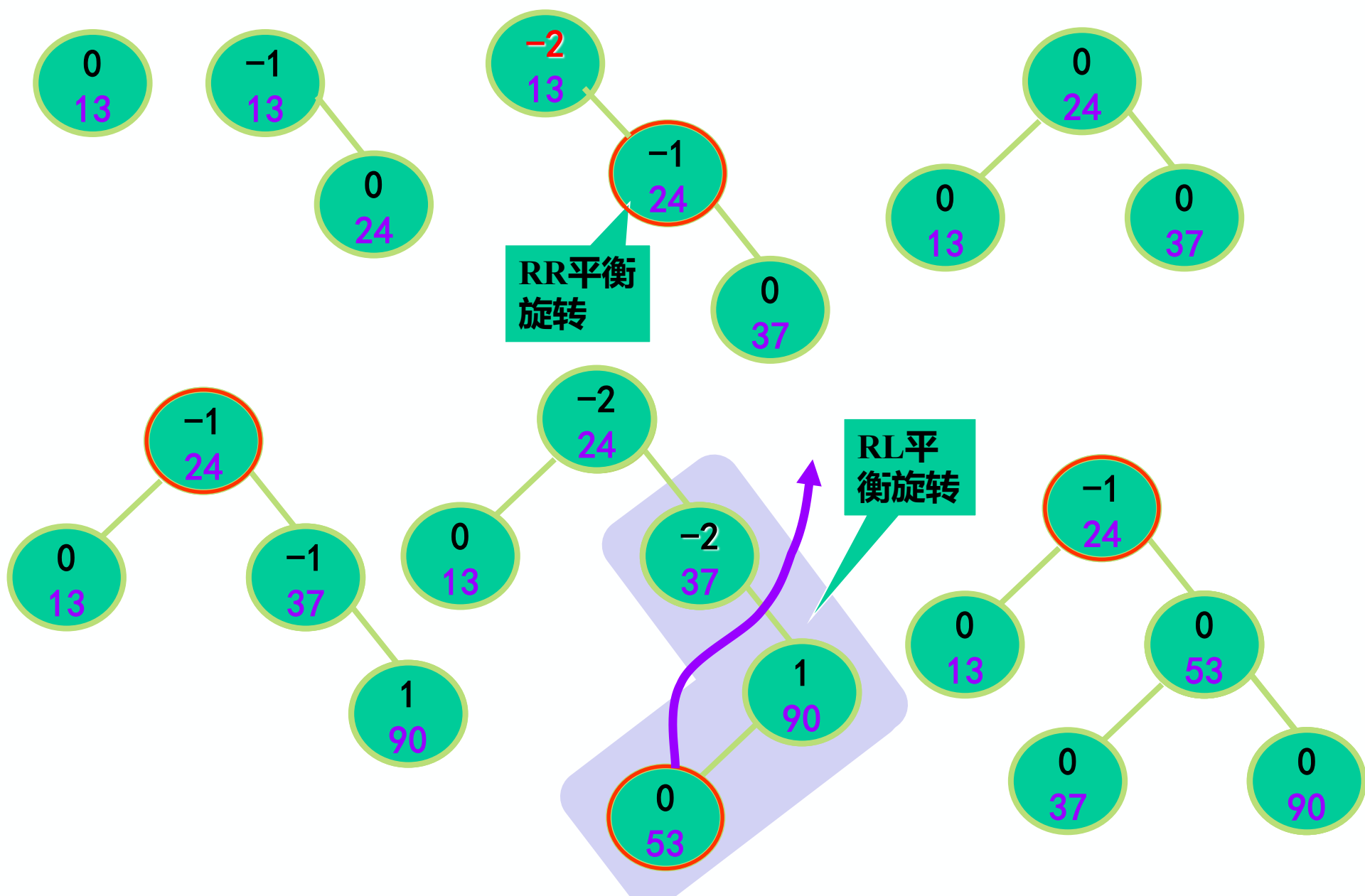
4) RL平衡旋转:



插入关键字8

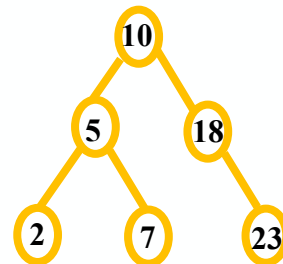
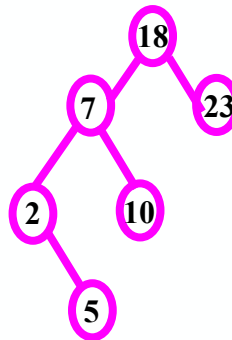
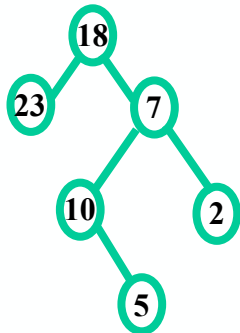
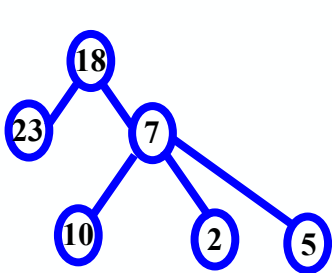


练习：请将下面序列构成一棵平衡二叉排序树
(13, 24, 37, 90, 53)



7.3.4 B树 (P259)

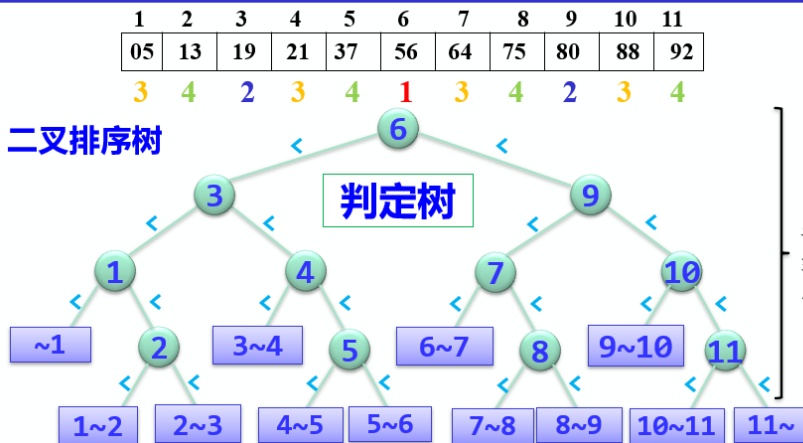
B树 == B-树 == B-tree == B tree



重点研究二叉树

✓结构简单，存储效率高，二叉树操作算法相对简单且任何树都很容易转化成唯一对应的二叉树。

折半查找



如何提高二叉排序树的查找效率?

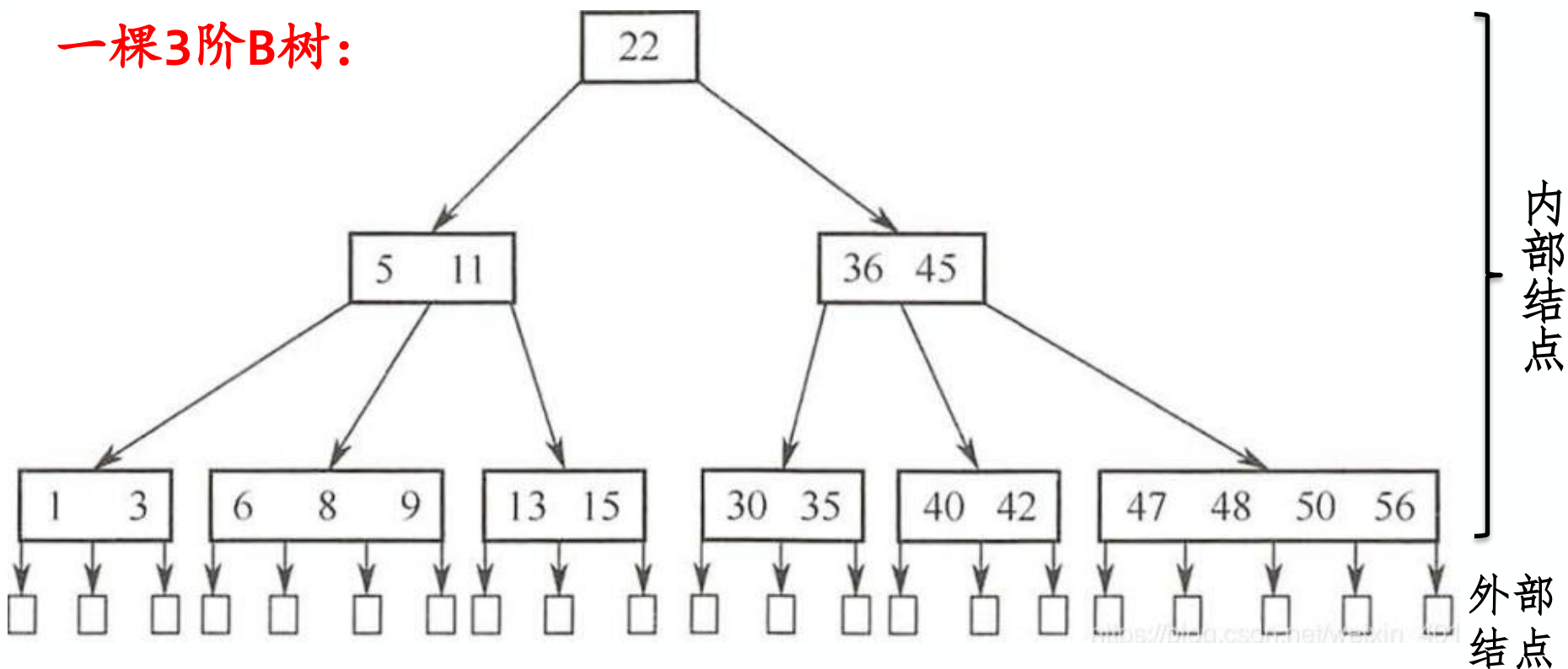
树的高度越小，查找速度越快。尽量让二叉排序树的形状均衡。

B树是对平衡二叉树AVL的推广，每个结点可以有多个子结点，并且每个结点可以存储多个关键字值，从而减少树的高度，提高搜索效率。使其适用于更大规模的数据存储和查找。

7.3.4 B树 (P259)

B树是一个m叉树，又称为多叉平衡查找搜索，每个结点不止有一个元素，也可能会有多个分叉，更加提高查找效率。

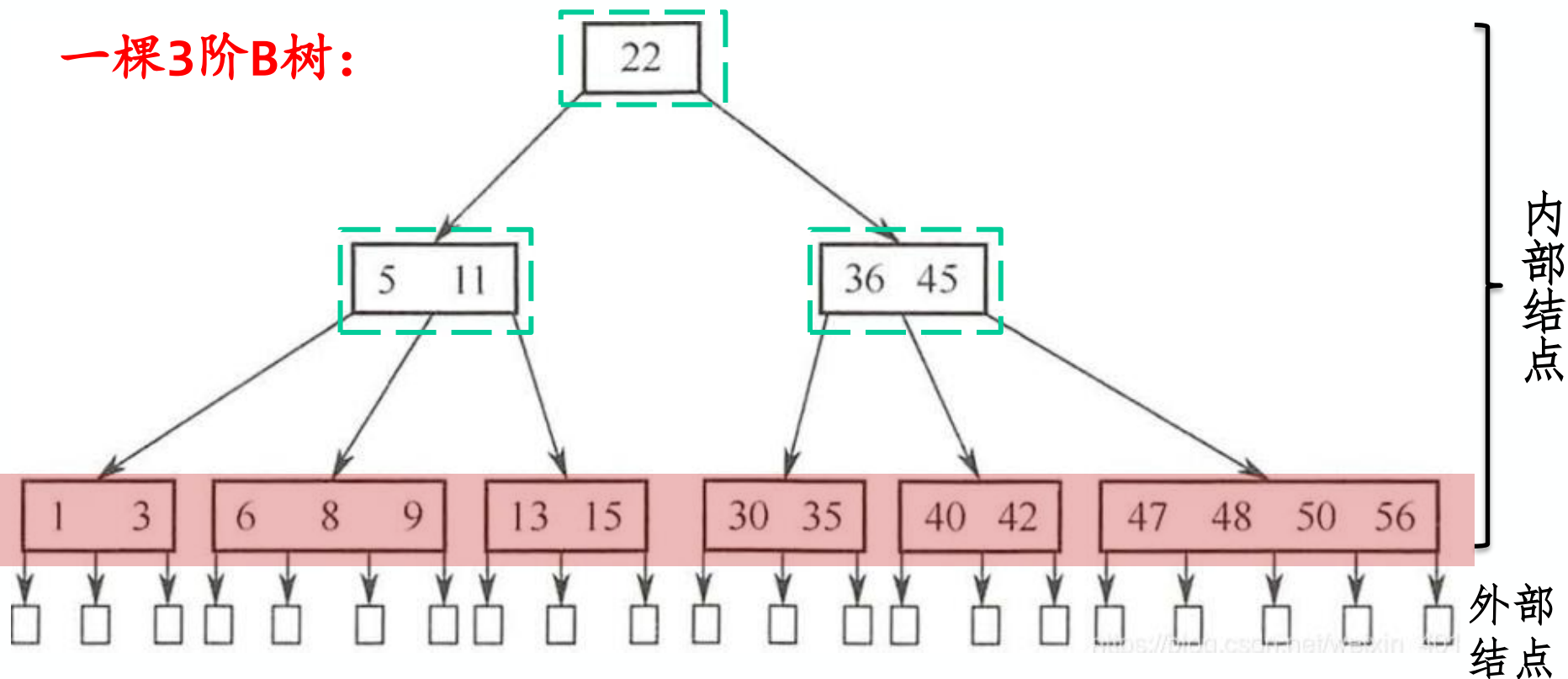
一棵3阶B树：



一棵m阶B树, 每个结点至多有m个孩子结点

7.3.4 B树 (P259)

一棵3阶B树:



平衡

所有的叶子结点
一定都在同一层

有序

结点内有序
任一元素的左子树都小
于它, 右子树都大于它

对于m阶B树:

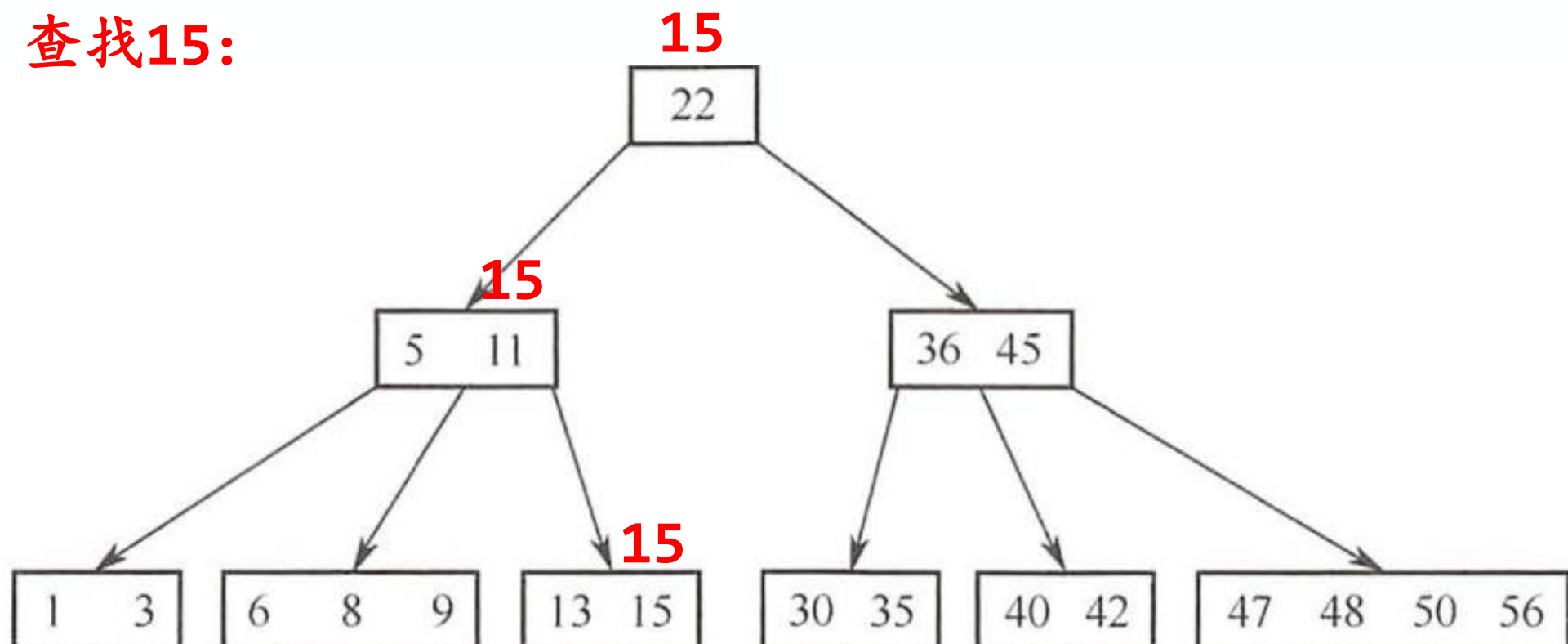
至多 m个分支, **多路**

非叶结点 m-1个关键字

至少 根结点: 2个分
支, 1个关键字

B树的查找

查找15:

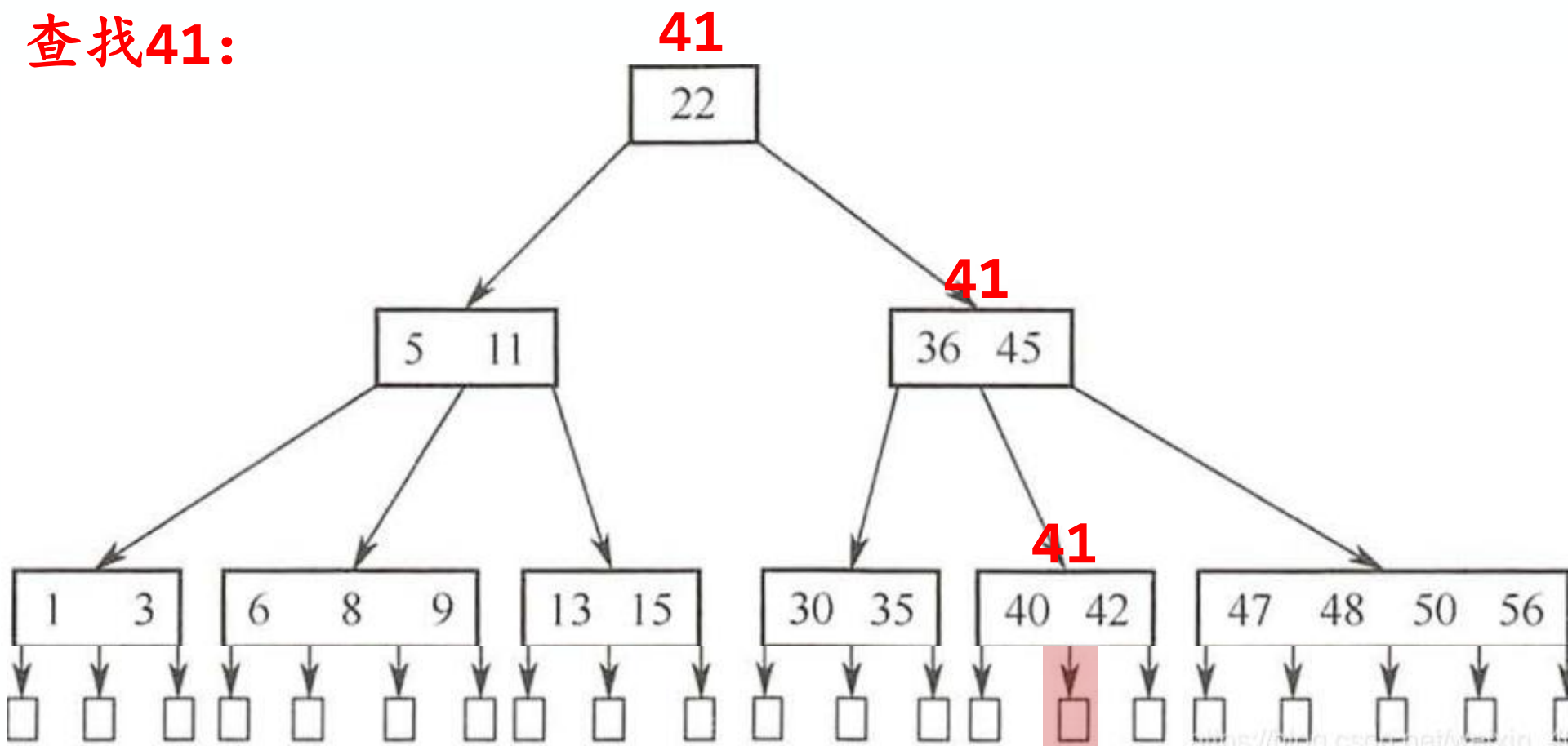


查找成功✓

因为结点内有多个元素，可采用顺序查找法依次对每个元素进行比较，比当前元素大，往后继续比，找到第一个比它小的元素，查其左子树；否则顺着该结点最右边指针链，针对其所指向的子结点继续查找。

B树的查找

查找41:

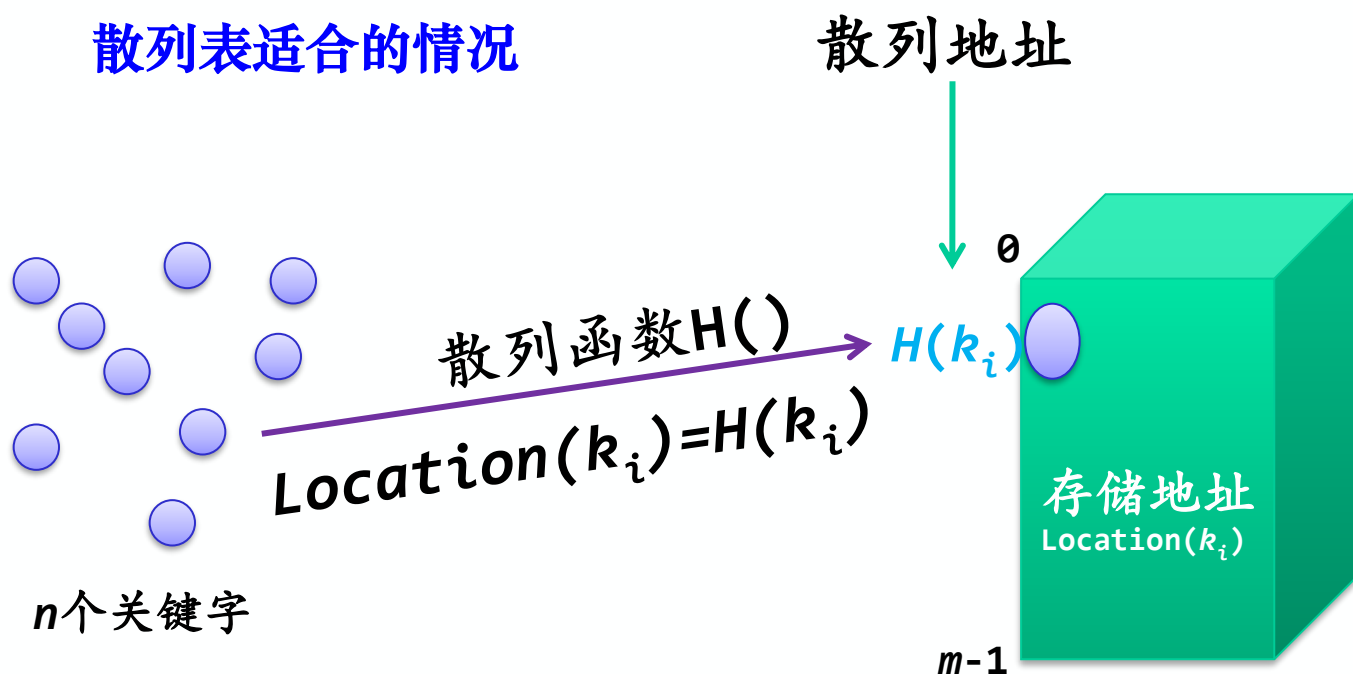


因为结点内有多个元素，可采用顺序查找法依次对每个元素进行比较，比当前元素大，往后继续比，找到第一个比它小的元素，查其左子树

7.4 散列表的基本概念(P268)

散列表(hash)又称为哈希表，是一种存储结构（物理结构），不适合杂乱无章或有序表的存储，适合关键字与存储地址存在某种函数关系的数据。

散列表适合的情况



散列表：有限连续的一维数组空间，用以存储按散列函数计算得到相应散列地址的关键字

散列函数：计算关键字 k_i 应该存放的散列地址

示例

对于如下9个关键字

{ Zhao, Sun, Li, Tang, Chen, Han, Ye, Dai, Bai}

设散列函数的值取关键字key中第一个字母在字母表中的序号,

即 $H(\text{key}) = \text{key的首字母} - 'A'$

A: $'A' - 'A' = 0$

B: $'B' - 'A' = 1$

0	1	2	3	...	7	...	11	...	18	19	...	24	25
An	Bai	Chen	Dai		Han		Li		Sun	Tang		Ye	Zhao

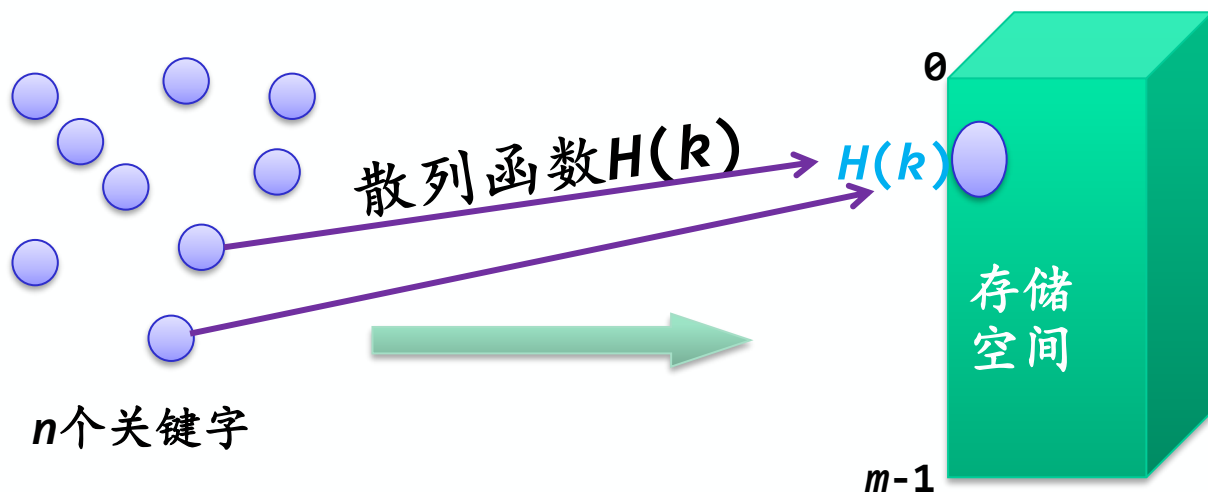
一个散列地址只能存放一个关键字

问题: 若添加关键字 Zhang, 会发生什么情况?

“冲突”

冲突：不同关键字映射到同一个散列地址

两个关键字 k_i 和 k_j ($i \neq j$) , $k_i \neq k_j$, 但 $H(k_i) = H(k_j)$ 。
即不同关键字 k_i 和 k_j 具有相同的函数值, 此现象叫做**冲突**。
其中具有相同函数值的多个关键字 k_i 和 k_j 互为**同义词**。



在散列查找方法中, 冲突是很难避免的! 只能尽可能减少!!

【练习6】 哈希表中出现同义词冲突是指（ ）。

- A. 两个元素具有相同的序号**
- B. 两个元素的关键字不同，而其他属性相同**
- C. 数据元素过多**
- D. 两个元素的关键字不同，而对应的哈希函数值 (存储地址、下标值) 相同**

答案：D

散列表设计

散列表的设计需要解决冲突。实际冲突是难以避免的，主要与3个因素有关：

0 1 2 3 ... 7 ... 11 ... 18 19 ... 24 25

An	Bai	Chen	Dai	Ege	Han	Jac	Li	Mo	Sun	Tang	We	Ye	Zhao
----	-----	------	-----	-----	-----	-----	----	----	-----	------	----	----	------

- 与装填因子有关。装填因子 α =存储的关键字个数/散列表长度= n/m
⇒若散列表长度固定，则存储的关键字越少，发生冲突的概率就越小，即比值 α 越小，冲突的可能性就越小，但空间利用率低，容易造成浪费； α 越大（最大可取1），说明散列表中关键字越多，表装得越满，越满发生冲突的可能性就越大，查找时比较的次数就越多。
通常使 α 控制在0.6~0.9的范围内。
- 与所采用的散列函数有关。好的散列函数会减少冲突的发生；不好的散列函数会增加冲突的发生。
- 与解决冲突方法有关。好的冲突解决方法会减少冲突的发生。

所以散列表设计的重点：

- 尽可能设计好的散列函数
- 找出解决冲突的方法。

构造散列函数考虑的因素

- ① 执行速度（即计算散列函数所需时间） 简单快速
- ② 关键字的长度 散列函数应能处理不同长度的关键字，对于长关键字，计算时间不应过长。
- ③ 散列表的长度 适中
- ④ 关键字的分布情况 均匀
- ⑤ 记录的查找频率 频繁访问的记录放在冲突较少的位置

设计好的散列函数：

- ①函数计算要尽可能简单，以便提高转换速度，每个关键字只能有一个散列地址与之对应
- ②函数的值域需在表长范围内，计算出的散列地址分布均匀，尽量减少冲突

构造散列函数的常用方法：

1) 直接定址法

2) 数字分析法

3) 平方取中法

4) 折叠法

5) 除留余数法

6) 随机数法

7.4.2 散列函数的构造方法 P270

1、直接定址法

直接定址法是以关键字 key 本身或关键字乘以某常量 a ，再加上某常量 b ，作为散列地址（函数值）的方法。

直接定址法的散列函数 $H(key)$ 为：

$$H(key) = key \text{ 或 } H(key) = a \times key + b$$

例如：{201001003、201001004、201001005、201001006、...}

$$H(\text{学号}) = \text{学号} - 201001001$$

优点：以关键字 key 的某个线性函数值作为散列地址，不会产生冲突

缺点：要占用连续的地址空间，空间效率低

7.4.2 散列函数的构造方法

2、数字分析法

若将学生信息按如下方式存入计算机，如

关键字

2	0	2	2	1	6	0	2
2	0	2	2	3	6	7	5
2	0	2	0	1	6	2	8
2	0	2	2	1	6	3	4
2	0	2	7	1	6	1	6
2	0	2	2	8	6	3	8
2	0	2	2	1	6	6	2
2	0	3	1	9	6	2	0

取最后
两位作
为散列
地址



0	2
7	5
2	8
3	4
1	6
3	8
6	2
2	0

散列地址的集合为(2, 75, 28, 34, 16, 38, 62, 20)

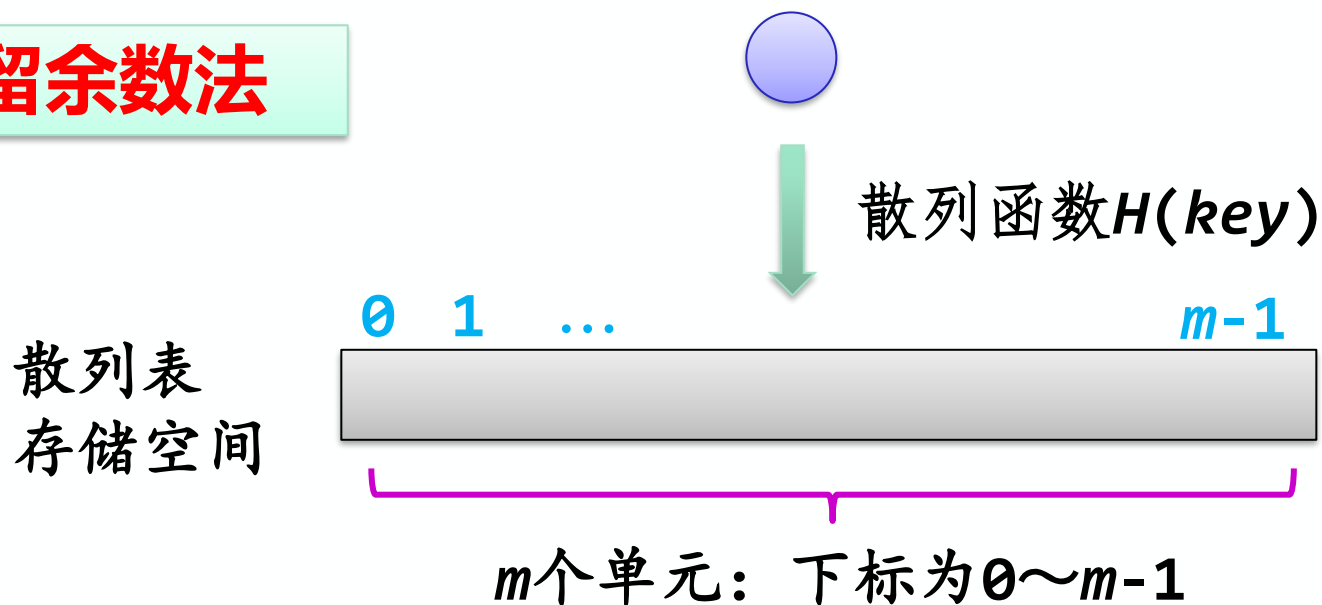
适用于大数值

散列函数 $H(k)$

小数值范围

7.4.2 散列函数的构造方法

3、除留余数法



除留余数法的散列函数 $H(key)$ 为：

$$H(key) = key \bmod p \quad (\text{mod 为取余运算})$$

p 最好是不大于表长 m 的最大素数（质数）。

除留余数法就是把 n 个记录按关键字映射的 $0 \sim m-1$ 的散列空间中，取余 p （素数）时出现冲突的可能性最小。除留余数法计算简单，适用范围广，是最常用的散列函数构造方法。

【例】假设散列表长度 $m=13$ ，采用除留余数法构造散列函数，建立如下关键字集合的散列表(16, 74, 60, 43, 54, 90, 46, 31, 29, 88, 77)。

解：关键字个数为11，散列表长度 $m=13$ ，设计除留余数法的散列函数为：

$$H(key) = key \bmod p$$

p 应为 $\leq$ 表长 m 的最大素数， $p=13$

$$H(key) = key \bmod 13$$

关键字: 16 74 60 43 54 90 46 31 29 88 77

$$H(16) = 16 \% 13 = 3 \quad H(43) = 43 \% 13 = 4 \quad H(46) = 46 \% 13 = 7$$

$$H(74) = 74 \% 13 = 9 \quad H(54) = 54 \% 13 = 2 \quad H(31) = 31 \% 13 = 5$$

$$H(60) = 60 \% 13 = 8 \quad H(90) = 90 \% 13 = 12 \quad H(29) = 29 \% 13 = 3$$

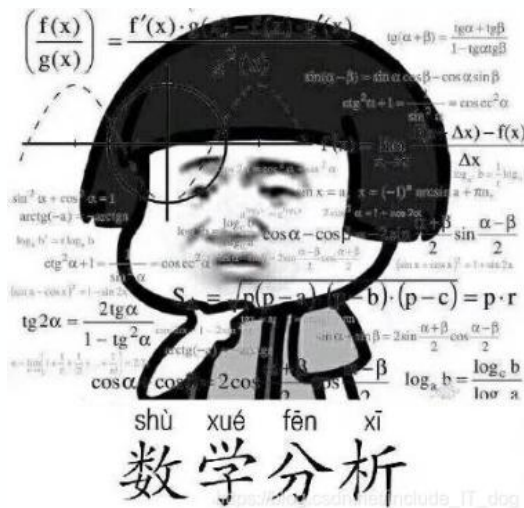
0	1	2	3	4	5	6	7	8	9	10	11	12
		54	16	43	31		46	60	74			90

注意: 存在冲突

好的散列函数可以一定程度上减少冲突,
但无法完全避免冲突的发生。

如何处理冲突?

有两种方法——开放地址法、链地址法



7.4.3 处理冲突的方法 P272

1、开放地址法

开放地址法：冲突时找下一个空闲的散列地址，此时所有数组空间对所有元素都是开放的。



怎么找空闲单元？

实例：电影院看电影迟到，找座位的情况就是采用开放定址法

如果你买了电影票，到电影院时已经开映了，你的位置被别人占用了，你需要找一个空位置。这就是开放定址法的思路。

7.4.3 处理冲突的方法 P272

开放地址法的公式:

$$H_i = (H(key) + \overset{\text{增量}}{d_i}) \bmod p$$

根据增量 d_i 取值的不同, 分为以下三种探测方法:

(1) 线性探测法

$$d_i = 1, 2, 3, \dots, p-1$$

(2) 二次探测法 (平方探测法)

$$d_i = 1^2, -1^2, 2^2, -2^2, \dots, k^2, -k^2 \quad (k \leq p/2)$$

(3) 伪随机探测法

d_i 是伪随机数序列

7.4.3 处理冲突的方法——开放定址法

(1) 线性探测法

线性探测法的数学递推描述公式为：

$$H_i = (H(key) + d_i) \bmod p$$

$$d_i = 1, 2, 3, \dots, p-1$$

非同义词冲突：第一个散列函数值不相同的两个关键字争夺同一个后继散列地址 $\Rightarrow$ **堆积（或聚集）现象**。

7.4.3 处理冲突的方法——线性探测法

【例】假设散列表长度 $m=13$ ，采用除留余数法散列函数建立如下关键字集合的散列表： $H(key)=key \bmod 13$
(16, 74, 60, 43, 54, 90, 46, 31, 29, 88, 77)
并采用线性探测法解决冲突。

关键字: ³16 ⁹74 ⁸60 ⁴43 ²54 ¹²90 ⁷46 ⁵31 29 88 77

$H(29)=29\%13=3 \Rightarrow$ 冲突 29和16是同义词

$H_1=(H(29)+1)\%13=4 \Rightarrow$ 堆积

$H_2=(H(29)+2)\%13=5 \Rightarrow$ 堆积

$H_3=(H(29)+3)\%13=6 \Rightarrow$ OK

0	1	2	3	4	5	6	7	8	9	10	11	12
		54	16	43	31	29	46	60	74			90

关键字29共探测4次

7.4.3 处理冲突的方法——线性探测法

【例】假设散列表长度 $m=13$ ，采用除留余数法散列函数建立如下11个关键字集合的散列表：

(16, 74, 60, 43, 54, 90, 46, 31, 29, 88, 77)

并采用线性探测法解决冲突。

关键字：16 74 60 43 54 90 46 31 29 88 77



$$H(88) = 88 \% 13 = 10 \quad \Rightarrow \text{OK}$$

0	1	2	3	4	5	6	7	8	9	10	11	12
		54	16	43	31	29	46	60	74	88		90

关键字88共探测1次

7.4.3 处理冲突的方法——线性探测法

【例】假设散列表长度 $m=13$ ，采用除留余数法散列函数建立如下11个关键字集合的散列表：

(16, 74, 60, 43, 54, 90, 46, 31, 29, 88, 77)

并采用线性探测法解决冲突。

关键字：16 74 60 43 54 90 46 31 29 88 77

$H(77)=12 \Rightarrow$ 冲突

$H_1=(H(77)+1)\%13=0 \Rightarrow$ OK

0	1	2	3	4	5	6	7	8	9	10	11	12
77		54	16	43	31	29	46	60	74	88		90

关键字77共探测2次，散列表创建完毕

7.4.3 处理冲突的方法——线性探测法

【例】假设散列表长度 $m=13$ ，采用除留余数法散列函数建立如下11个关键字集合的散列表：

(16, 74, 60, 43, 54, 90, 46, 31, 29, 88, 77)

并采用线性探测法解决冲突。

最终的散列表

散列表hash[0..12]

下标	0	1	2	3	4	5	6	7	8	9	10	11	12
key	77		54	16	43	31	29	46	60	74	88		90
探测次数	2		1	1	1	1	4	1	1	1	1		1



散列表的构成：

- 散列函数：本例为 $H(key)=key \bmod 13$
- 解决冲突方法：本例为线性探测法

7.4.3 处理冲突的方法——开放定址法

成功查找的情况

$$H(29) = 29 \% 13 = 3: 16 \neq 29;$$

查找关键字为 29 的记录: $H_1 = (H(29) + 1) \% 13 = 4: 43 \neq 29;$

$$H_2 = (H(29) + 2) \% 13 = 5: 31 \neq 29;$$

$$H_3 = (H(29) + 3) \% 13 = 6: 29 = 29。成功！$$

探测次数恰好等于查找关键字的比较次数！

需要 4 次关键字比较

下标	0	1	2	3	4	5	6	7	8	9	10	11	12
key	77		54	16	43	31	29	46	60	74	88		90
探测次数	2		1	1	1	1	4	1	1	1	1		1

散列表成功查找完毕

7.4.3 处理冲突的方法——开放定址法

❶ 对于前面构建的散列表：成功查找 $ASL_{成功}$ 计算

散列表hash[0..12]

下标	0	1	2	3	4	5	6	7	8	9	10	11	12
key	77		54	16	43	31	29	46	60	74	88		90
比较次数	2		1	1	1	1	4	1	1	1	1		1

探测次数恰好等于查找关键字的比较次数！



$$ASL_{成功} = \frac{2+1+1+1+1+4+1+1+1+1+1+1}{11} = 1.364$$

7.4.3 处理冲突的方法——开放定址法

不成功查找的情况

查找关键字**47**的记录

$$H(47) = 47 \% 13 = 8$$

$$\Rightarrow 60 \neq 47$$

$$H_1 = (H(47) + 1) \% 13 = 9$$

$$\Rightarrow 74 \neq 47$$

$$H_2 = (H(47) + 2) \% 13 = 10$$

$$\Rightarrow 88 \neq 47$$

$$H_3 = (H(47) + 3) \% 13 = 11$$

此处为**空**，**查找失败!**
47一定不在表中存在

下标	0	1	2	3	4	5	6	7	8	9	10	11	12
key	77		54	16	43	31	29	46	60	74	88		90
探测 次数	2		1	1	1	1	4	1	1	1	1		1

需要**4**次关键字比较

查找失败

7.4.3 处理冲突的方法——开放定址法

❶ 对于前面构建的散列表：失败查找 $ASL_{\text{失败}}$ 计算

散列表hash[0..12]

下标	0	1	2	3	4	5	6	7	8	9	10	11	12
key	77		54	16	43	31	29	46	60	74	88		90
比较 次数													

查找14 $H(14)=14\%13=1$

查找28 $H(28)=28\%13=2$

一个关键字不在有效关键字集合中，但其散列函数值为2，
采用线性探测法找到空位置，需要10次关键字比较



$$ASL_{\text{不成功}} = \frac{2+1+10+9+8+7+6+5+4+3+2+1+3}{13} = 4.692$$

7.4.3 处理冲突的方法——开放定址法

(2) 平方探测法（二次探测法）

平方探测法的数学描述公式为：

$$H_i = (H(key) + d_i) \bmod m$$

$$d_i = 1^2, -1^2, 2^2, -2^2 \dots, q^2$$

查找的位置依次为： $H(key)$ 、 $H(key)+1$ 、 $H(key)-1$ 、 $H(key)+4$ 、 $H(key)-4$ 、 $H(key)+9$ 、 $H(key)-9$...

示例：在电影院中找被占用位置左右的空位置！

平方探测法是一种较好的处理冲突的方法，可以减少出现**堆积现象**。它的缺点是不能探测到散列表上的所有单元，但至少能探测到一半单元。

7.4.3 处理冲突的方法——平方探测法

【例】选取散列函数 $H(k) = (k) \text{ MOD } 11$ 。用二次探测再散列处理冲突，试在 $0 \sim 10$ 的散列地址空间中对关键字序列 $(22, 41, 53, 46, 30, 13, 01, 67)$ 构造散列表，并求等概率情况下查找成功时的平均查找长度。

关键字: 22 41 53 46 30 13 01 67

$H(30) = 30 \% 11 = 8$ **⇒ 冲突**

$$H_1 = (8+1) \% 11 = 9 \quad \Rightarrow \text{堆积}$$
$$H_2 = (8-1) \% 11 = 7 \quad \Rightarrow \text{OK}$$

0	1	2	3	4	5	6	7	8	9	10
22		46					30	41	53	

共探測3次

7.4.3 处理冲突的方法——平方探测法

【例】选取散列函数 $H(k) = (k) \text{ MOD } 11$ 。用二次探测再散列处理冲突，试在 $0 \sim 10$ 的散列地址空间中对关键字序列 $(22, 41, 53, 46, 30, 13, 01, 67)$ 构造散列表，并求等概率情况下查找成功时的平均查找长度。

关键字: 22 41 53 46 30 13 01 67

$H(13) = 13 \% 11 = 2$ **⇒ 冲突**

$H_1 = (2+1) \% 11 = 3$ **⇒ OK**

0	1	2	3	4	5	6	7	8	9	10
22		46	13				30	41	53	

共探測2次

7.4.3 处理冲突的方法——平方探测法

【例】选取散列函数 $H(k) = (k) \text{ MOD } 11$ 。用二次探测再散列处理冲突，试在 $0 \sim 10$ 的散列地址空间中对关键字序列 $(22, 41, 53, 46, 30, 13, 01, 67)$ 构造散列表，并求等概率情况下查找成功时的平均查找长度。

关键字: 22 41 53 46 30 13 01 67

$H(67) = 67 \% 11 = 1 \Rightarrow \text{冲突}$

$H_1 = (1+1) \% 11 = 2$ $\Rightarrow$ 堆积

$H_2 = (1-1) \% 11 = 0 \quad \Rightarrow \text{堆积}$

H₃=(1+4) % 11=5 **⇒ OK**

0	1	2	3	4	5	6	7	8	9	10
22	01	46	13		67		30	41	53	

共探測4次

7.4.3 处理冲突的方法——平方探测法

❶ 对于前面构建的散列表：成功查找 $ASL_{成功}$ 计算

散列表hash[0..10]

下标	0	1	2	3	4	5	6	7	8	9	10
key	22	01	46	13		67		30	41	53	
探测次数	1	1	1	2		4		3	1	1	

探测次数恰好等于查找到该记录所需要的关键字比较次数！

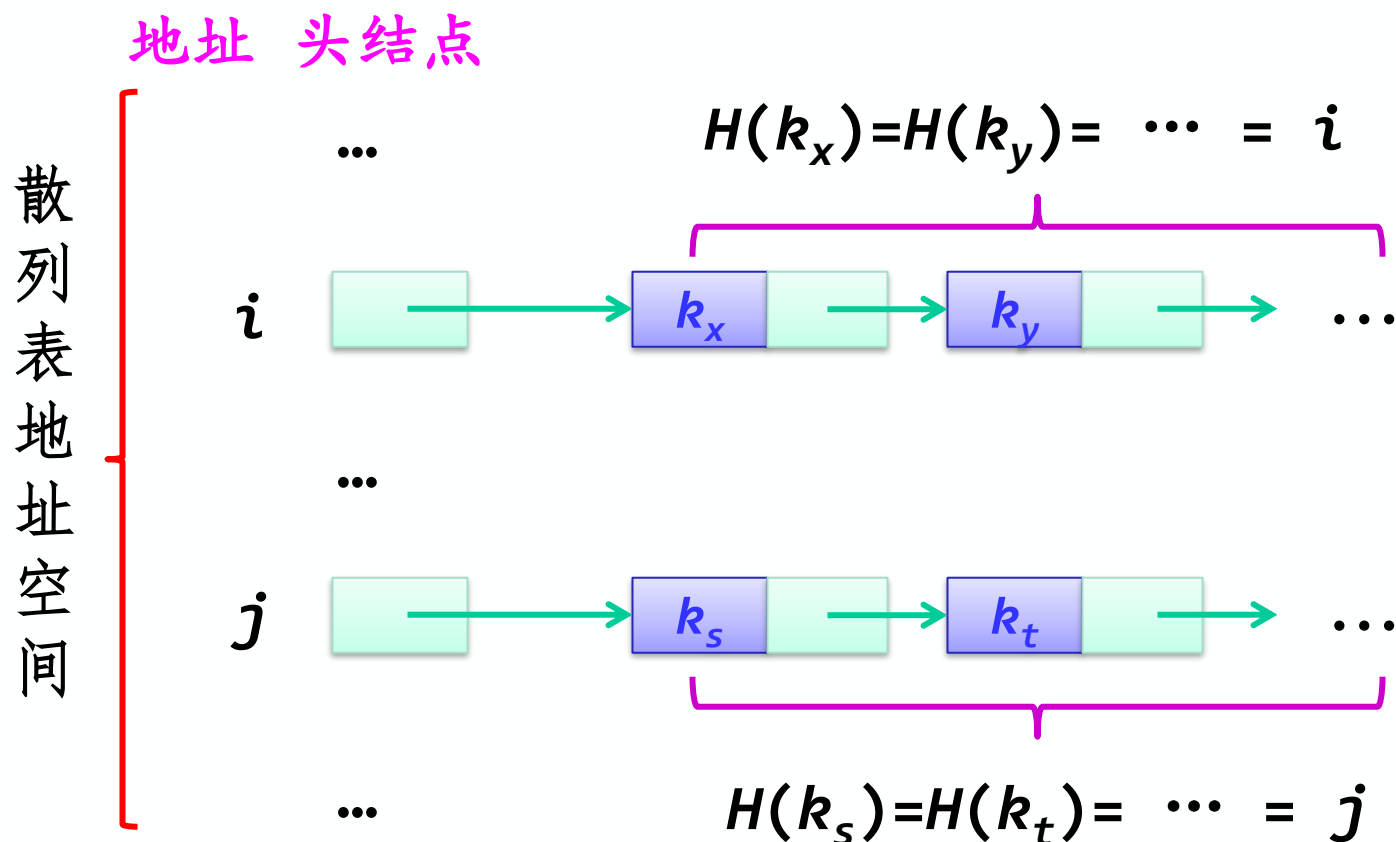


$$ASL_{成功} = \frac{1+1+1+2+4+3+1+1}{8} = 1.75$$

7.4.3 处理冲突的方法 P217

2、链地址法

也称为拉链法，是把所有的同义词用单链表链接起来的方法。
头结点记录散列地址，链表的数据域存放对应的关键字



表对应m个散列地址，就设置m个单链表

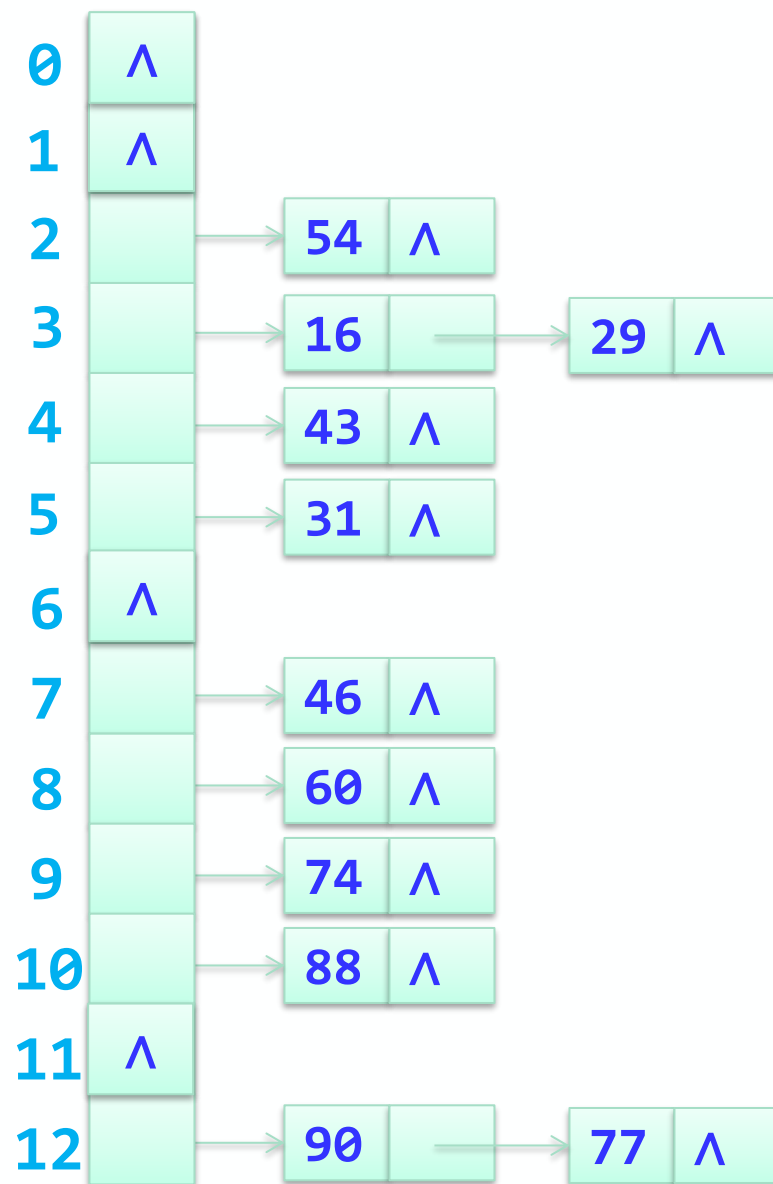
7.4.3 处理冲突的方法——链地址法

【例】构造采用**拉链法**解决冲突的散列表。

$$H(key) = key \bmod 13$$

假设散列表长度 $m=13$,
如下散列表关键字用除留
余数法作为散列函数:
(16, 74, 60, 43, 54,
90, 46, 31, 29, 88,
77)

采用拉链法构造的散列表hash[0...12]



练习一下

1. 顺序查找法适合于存储结构为（ **B** ）的线性表。
A. 散列存储 B. 顺序存储或链式存储
C. 压缩存储 D. 索引存储
2. 对线性表进行折半查找时，要求线性表必须（ **B** ）。
A. 以顺序方式存储
B. 以顺序方式存储，且结点按关键字有序排列
C. 以链式方式存储
D. 以链式方式存储，且结点按关键字有序排列
3. 有一个有序表为{1,3,9,12,32,41,45,62,75,77,82,95,100}，当折半查找值82的元素时，（ **D** ）次比较后查找成功。
A. 1 B. 2 C. 3 D. 4

1	2	3	4	5	6	7	8	9	10	11	12	13
1	3	9	12	32	41	45	62	75	77	82	95	100

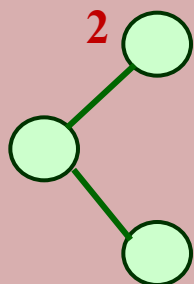
练习一下

1. 二叉树为二叉排序树的充要条件是其任一结点的值均大于其左子树的值，小于其右子树的值。这种说法（ A ）。
A. 正确 B. 错误
2. 在一棵平衡二叉树中，每个结点的平衡因子的取值范围是（ A ）。
A. $-1 \sim 1$ B. $-2 \sim 2$ C. $1 \sim 2$ D. $0 \sim 1$
3. 查找效率最高的二叉排序树是（ C ）。
A. 所有结点的左子树都为空的二叉排序树
B. 所有结点的右子树都为空的二叉排序树
C. 平衡二叉树
D. 没有左子树的二叉排序树

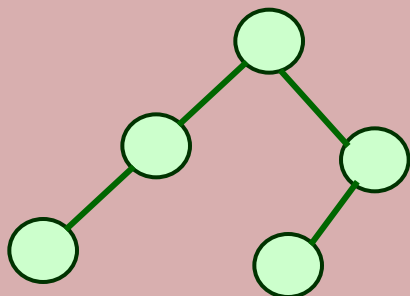
4. (**B**) 遍历一棵二叉排序树所得的结点访问序列是按结点值的递增序列。

A. 先序 B. 中序 C. 后序 D. 以上均不是

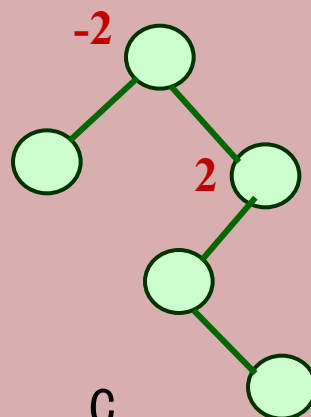
5. 如图所示的4棵二叉树, (**B**) 是平衡二叉树。



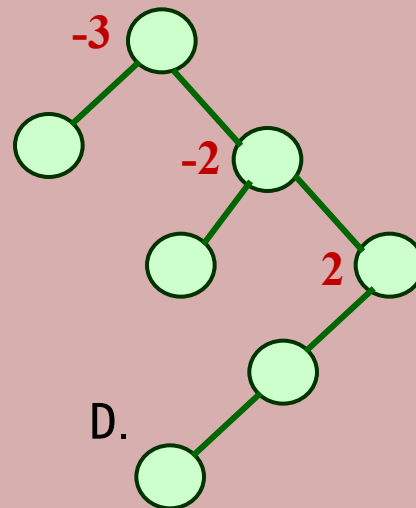
A.



B.

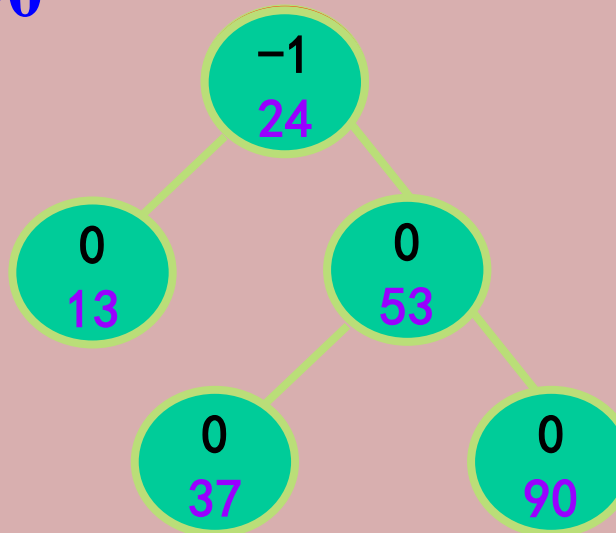


C.



D.

6. 按13, 24, 37, 90, 53的次序形成平衡二叉树, 则该平衡二叉树的高度是 3, 其根是 24, 左子树的数据是 13; 右子树的数据是 37 53 90



7. 4阶的B-树上所有非终端结点至多可有 (3) 个关键字。

练习一下

以下说法错误的是（ **B** ）。

- A. 散列表存储的基本思想是由关键字值决定数据的存储地址。
- B. 散列表的结点中只包含数据元素自身的信息，不包含任何指针。 链地址法处理冲突，包含关键字所在的数据域和指针域
- C. 装填因子是散列表的一个重要参数，它反映了散列表的装填程度。
- D. 散列表的查找效率主要取决于散列表构造时选取的散列函数和处理冲突的方法。

散列表的平均查找长度（ **C** ）。

- A. 与处理冲突方法有关而与表的长度无关
- B. 与处理冲突方法无关而与表的长度有关
- C. 与处理冲突方法有关且与表的长度有关
- D. 与处理冲突方法无关且与表的长度无关

设哈希表表长 $m=14$ ，哈希函数 $H(\text{key})=\text{key}\%11$ 。表中已有4个结点： $H(15)=4$ ， $H(38)=5$ ， $H(61)=6$ ， $H(84)=7$ ，其余地址为空。若用**二次探测**再散列处理冲突，关键字为49的结点的地址是（ **D** ）。

- A. 8 B. 3 C. 5 D. 9

$$H(49)=49\%11=5$$

$$H(49)=(5-1)\%11=4$$

$$H(49)=(5+1)\%11=6$$

$$H(49)=(5+4)\%11=9$$

散列函数有一个共同性质即函数值应按（C）取其值域的每一个值。

A. 最大概率 B. 最小概率 C. 同等概率 D. 随机概率